

Python Basics

Instructions	Commands/orders that we give to the computer.
Program	Set of instructions, executed by the computer to perform a specific task.
Language	Mode of communication b/w two people. E.g., We use languages like English/Hindi to communicate with each other.
Programming Language	- Programming languages are used for communication b/w humans (programmers) & computers. OR It is a formal set of rules & syntax used to communicate with computers & instruct them to perform a specific task. - Each programming language has its own unique syntax → the rules & structure for writing instructions. - Programmers use these instructions to give commands to the computer, which is then executed to carry out a task. E.g., Programmers use programming languages (C, C++, Java, Python) to communicate with computers.
IDE (Integrated Development Environment)	Software that provides an environment/platform to write, run & test any programming language .
IDLE (Integrated Development and Learning Environment)	- Software that provides an environment/platform specifically designed to write, run & test Python programs. - It includes a text editor (for writing code) & an interpreter (to execute Python commands and see results immediately).

Module 1

(1) List

Methods	Description
append (item)	Adds a single item to the end of the list (takes one argument).
extend (iterable)	Adds all elements of an iterable (list, tuple or string) to the end of the list.
insert (index, item)	Inserts an item at the specified index. Shifts existing elements to the right.
remove (item)	Removes the first occurrence of the specified item.
pop () pop (index)	Removes & returns the last element of the list. Removes & returns the element at the specified index.
clear ()	Removes all items from the list, making it empty.
index (item)	Returns the index of the first occurrence of the specified item.
count (item)	Returns the no. of times the specified item appears in the list.
sort () sort(reverse=True)	Sorts the list in ascending order . Sorts the list in descending order.
reverse ()	Reverses the order of elements in the list.
copy ()	Returns a shallow copy (new list) of the original list. Original remains unchanged.

Methods that do not modify the original list

- index(item)
- count(item)
- copy ()

```

# Create
list = ["apple", "banana", "cherry"] # ['apple', 'banana', 'cherry']

# Access
print ("Access index 2:", my_list[2]) # cherry

# Update / Replace
list [1] = 'orange' # ['apple', 'orange', 'cherry']
list [2] = ['banana'] # ['apple', 'orange', ['banana']]
list [2] = 5 # ['apple', 'orange', 5]
list [2] = '5' # ['apple', 'orange', '5']
list [2] = [5] # ['apple', 'orange', [5]]
list [2] = ['5'] # ['apple', 'orange', ['5']]
list [2] = [banana] # NameError: name 'banana' is not defined

# Append
list.append('orange') # ['apple', 'banana', 'cherry', 'orange']
list.append(['grapes']) # ['apple', 'banana', 'cherry', ['grapes']]
list.append(5) # ['apple', 'banana', 'cherry', 5]
list.append('5') # ['apple', 'banana', 'cherry', '5']
my_list.append([5]) # ['apple', 'banana', 'cherry', [5]]
list.append(['5']) # ['apple', 'banana', 'cherry', ['5']]
list.append([1, 2, 3, 4, 5]) # ['apple', 'banana', 'cherry', [1, 2, 3, 4, 5]]

list.append(orange) # NameError: name 'orange' is not defined. Did you mean: 'range'?
list.append('orange', 'mango') # TypeError: list.append() takes exactly one
argument (2 given)

# Extend
list.extend('orange') # ['apple', 'banana', 'cherry', 'o', 'r', 'a', 'n', 'g',
'e']
list.extend(['grapes']) # ['apple', 'banana', 'cherry', 'grapes']
list.extend(['grapes', 'guava', 'mango']) # ['apple', 'banana', 'cherry',
'grapes', 'guava', 'mango']
list.extend([5]) # ['apple', 'banana', 'cherry', 5]
list.extend(['5']) # ['apple', 'banana', 'cherry', '5']
list.extend([1, 2, 3, 4, 5]) # ['apple', 'banana', 'cherry', 1, 2, 3, 4, 5]

list.extend(5) # TypeError: 'int' object is not iterable

# Insert
list.insert(1, "mango") # ['apple', 'mango', 'banana', 'cherry']
list.insert(1, 25) # ['apple', 25, 'banana', 'cherry']
list.insert(1, ["guava"]) # ['apple', ['guava'], 'banana', 'cherry']

list.insert (1, mango) # NameError: name 'mango' is not defined. Did you mean: 'range'?

```

```
# Remove
list = ["apple", "banana", "cherry", "banana"]
list.remove("banana") # ['apple', 'cherry', 'banana']

# Pop
list = ["apple", "banana", "cherry", "banana"]
list.pop(1) # ['apple', 'cherry', 'banana']
list.pop() # ['apple', 'cherry']

# Clear
list.clear() # []

# Index & Count
list = ["apple", "banana", "cherry", "banana"]
list.index("banana") # 1
list.count("banana") # 2
list.count("mango") # 0

# Sort
list1 = ["cherry", "banana", "apples", "banana", "apple"]
list2 = [2, 1, 5, 3, 4]

list1.sort() # ['apple', 'apples', 'banana', 'banana', 'cherry']
list2.sort() # [1, 2, 3, 4, 5]
list1.sort(reverse=True) # ['cherry', 'banana', 'banana', 'apples', 'apple']
list2.sort(reverse=True) # [5, 4, 3, 2, 1]

# Reverse
list1 = ["cherry", "banana", "apples", "banana", "apple"]
list2 = [2, 1, 5, 3, 4]
list1.reverse() # ['apple', 'banana', 'apples', 'banana', 'cherry']
list2.reverse() # [4, 3, 5, 1, 2]

print (list1) # TypeError: list.reverse() takes no keyword arguments

# Copy
list1 = ["apple", "banana", "cherry"]
list2 = list1.copy()
list1.append('mango')

print(list1) # ['apple', 'banana', 'cherry', 'mango']
print(list2) # ['apple', 'banana', 'cherry']
```

```

# Exercise
# 1. Create a list of 6 fruits.
#     - Print the first element using positive indexing.
#     - Print the last element using negative indexing.
#     - Slice the list to get the 2nd, 3rd, and 4th elements.
#     - Slice to get every second element from the list.
fruits = ["apple", "banana", "cherry", "date", "elderberry", "fig"]
print("First element (positive indexing):", fruits[0])
print("Last element (negative indexing):", fruits[-1])
print("2nd, 3rd, and 4th elements:", fruits[1:4])
print("Every second element:", fruits[::2])
print()

# 2. Given:
#     nums = [5, 10, 15, 20, 25, 30, 35, 40]
#     - Print the first four elements.
#     - Print the last four elements.
#     - Reverse the list using slicing.
#     - Slice the list from index 2 to 6 (exclusive).
nums = [5, 10, 15, 20, 25, 30, 35, 40]
print("First four elements:", nums[:4])
print("Last four elements:", nums[-4:])
print("Reversed list:", nums[::-1])
print("Slice from index 2 to 6 (exclusive):", nums[2:6])
print()

# 3. colors = ["red", "green", "blue", "yellow"]
#     - Change the second element to "purple".
#     - Replace the last two elements with ["black", "white"].
#     - Insert "orange" at index 1.
#     - Append "pink" to the end.
#     - Extend the list with ["brown", "gray"].
colors = ["red", "green", "blue", "yellow"]
colors[1] = "purple" # Change second element
colors[-2:] = ["black", "white"] # Replace last two elements
colors.insert(1, "orange") # Insert 'orange' at index 1
colors.append("pink") # Append 'pink' to the end
colors.extend(["brown", "gray"]) # Extend list
print("Modified colors list:", colors)
print()

# 4. numbers = [1, 2, 3, 4, 5]
#     - Append 6 to the list.
#     - Extend the list with [7, 8, 9].
#     - Remove the number 3.
#     - Pop the first element.
#     - Pop the last element and store it in a variable.

```

```

numbers = [1, 2, 3, 4, 5]
numbers.append(6) # Append 6
numbers.extend([7, 8, 9]) # Extend list
numbers.remove(3) # Remove number 3
numbers.pop(0) # Pop first element
last_element = numbers.pop() # Pop last element and store
print("Modified numbers list:", numbers)
print("Popped last element:", last_element)
print()

# 5. Given:
#   data = [12, 45, 7, 23, 9]
#   - Find the length of the list.
#   - Find the maximum and minimum values.
#   - Find the sum of all elements.
#   - Sort the list.
data = [12, 45, 7, 23, 9]
print("Length:", len(data))
print("Max value:", max(data))
print("Min value:", min(data))
print("Sum of elements:", sum(data))
data.sort()
print("Sorted list:", data)
print()

# 6. Write a program to count how many times "cat" appears in:
#   animals = ["cat", "dog", "cat", "rabbit", "cat", "dog"]
animals = ["cat", "dog", "cat", "rabbit", "cat", "dog"]
cat_count = animals.count("cat")
print("'cat' appears", cat_count, "times")
print()

# 7. Given:
#   cities = ["London", "Paris", "Tokyo", "New York"]
#   - Print each city using a for loop.
#   - Print each city in uppercase.
#   - Print the length of each city's name.
cities = ["London", "Paris", "Tokyo", "New York"]
print("Cities:")
for city in cities:
    print(city)
print("Cities in uppercase:")
for city in cities:
    print(city.upper())
print("Length of each city's name:")
for city in cities:
    print(len(city))
print()

```

```
# 8. Write a program that takes:
#     marks = [80, 55, 70, 45, 90]
#     - Print "Pass" for marks >= 60 and "Fail" otherwise.
marks = [80, 55, 70, 45, 90]
for mark in marks:
    if mark >= 60:
        print(f"Mark {mark}: Pass")
    else:
        print(f"Mark {mark}: Fail")
print()

# 9. Create a list of numbers from 1 to 20.
#     - Slice to get numbers from index 5 to 15.
#     - Replace all even numbers in that slice with 0.
numbers_1_to_20 = list(range(1, 21))
slice_5_to_15 = numbers_1_to_20[5:16]
print("Slice from index 5 to 15:", slice_5_to_15)

# Replace even numbers in the slice with 0
for i in range(5, 16):
    if numbers_1_to_20[i] % 2 == 0:
        numbers_1_to_20[i] = 0
print("List after replacing even numbers in slice with 0:")
print(numbers_1_to_20)
print()

# 10. Given:
#     words = ["python", "java", "c++", "ruby", "go"]
#     - Reverse the list
words = ["python", "java", "c++", "ruby", "go"]
words.reverse()
print("Reversed list:", words)
```

(2) Tuple

Methods	Description
count (item)	Returns the no. of times the specified item appears in the tuple.
index (item)	Returns the index of the first occurrence of the specified item.

Extra Methods	Description
len (tuple)	Returns the no. of elements in the tuple.
min (tuple)	Returns the smallest element in the tuple.
max (tuple)	Returns the largest element in the tuple.
sum (tuple)	Returns the sum of all numeric elements in the tuple.
sorted (tuple)	Returns a new sorted list (ascending by default) from the tuple's elements. Does not modify the tuple.

Note

- Tuples **cannot be modified** – so none of their operations change them.
- If any operation looks like it changes a tuple, it's actually returning a **new object**, like a list.

```
# Creating a tuple
tuple = (10, 20, 30, 40, 20, 50) # (10, 20, 30, 40, 20, 50)

# 1. Access elements by index
tuple[0] # 10

# 2. Length
len(tuple) # 6

# 3. Count
tuple.count(20) # 2

# 4. Index
tuple.index(30) # 2

# 5. Membership Test
print("20 in tuple:", 20 in tuple) # True
print("60 in tuple:", 60 in tuple) # False

# 6. Concatenation
tuple2 = (60, 70)
tuple3 = tuple + tuple2 # (10, 20, 30, 40, 20, 50, 60, 70)

# 7. Repetition
tuple * 2 # (10, 20, 30, 40, 20, 50, 10, 20, 30, 40, 20, 50)

# 8. min, max, sum
min(tuple) # 10
max(tuple) # 50
sum(tuple) # 170

# 9. Sorted
a = sorted(tuple)
print(a) # returns list --> [10, 20, 20, 30, 40, 50]
print (tuple)
b = sorted(tuple, reverse=True)
print (b) # returns list --> [50, 40, 30, 20, 20, 10]
print(tuple)

# 10. Immutability
tuple[0] = 100 # TypeError: 'tuple' object does not support item assignment
```

```

letters = ("a", "b", "c")
numbers = (1, 2, 3)

# Nested Tuple
nested_tuple = (letters, numbers) # (('a', 'b', 'c'), (1, 2, 3))

# String to Tuple
string_to_tuple = tuple("horse") # ('h', 'o', 'r', 's', 'e')

# List to Tuple
list_to_tuple = tuple(["h", "o", "r", "s", "e"]) # ('h', 'o', 'r', 's', 'e')

# Concatenate
concatenate = string_to_tuple + tuple("power") # ('h', 'o', 'r', 's', 'e', 'p', 'o', 'w', 'e', 'r')

# Add list to tuple by converting
final_tuple = tuple(list_to_tuple) + ('p', 'o', 'w', 'e', 'r') # ('h', 'o', 'r', 's', 'e', 'p', 'o', 'w', 'e', 'r')
final_tuple = tuple((((("h", "o", "r", "s", "e"))))) + ('p', 'o', 'w', 'e', 'r') # ('h', 'o', 'r', 's', 'e', 'p', 'o', 'w', 'e', 'r')

final_tuple = tuple(tuple(["h", "o", "r", "s", "e"])) + ('p', 'o', 'w', 'e', 'r') # ('h', 'o', 'r', 's', 'e', 'p', 'o', 'w', 'e', 'r')

final_tuple = tuple(tuple("h", "o", "r", "s", "e")) + ('p', 'o', 'w', 'e', 'r') # TypeError: tuple expected
at most 1 argument, got 5

# Add string to tuple
result = tuple("horse") + "def" # TypeError: can only concatenate tuple
(not "str") to tuple

# Empty & Basic Tuples
a = ()
b = ("h", "o", "r", "s", "e")
type(a) # <class 'tuple'>
type(b) # <class 'tuple'>

# Tuples with Nesting
air_force = ("f12", "f15a", "f25a")
fighter_jets = (1988, 2005, 2016, air_force) # (1988, 2005, 2016, ('f12', 'f15a', 'f25a'))

```

(3) Sets

Methods	Shortcut	Description
add(item)		Adds a single item to the set.
union (set2)		Return a new set ($s1 \cup s2$).
update (set2)	=	Modifies original set ($s1 \cup s2$).
clear ()		Removes all elements from the set.
copy()		Returns a shallow copy of the set.
intersection(set2)	&	Returns a new set ($s1 \cap s2$).
intersection_update (set2)	&=	Modifies the original set ($s1 \cap s2$).
difference (set2)		Returns a new set ($s1 - s2$).
difference_update (set2)	-=	Modifies the original set ($s1 - s2$).
symmetric_difference (set2)	^	Returns a new set ($s1 \Delta s2$).
symmetric_difference_update (set2)	^=	Modifies the original set ($s1 \Delta s2$).
discard (item)		- Removes item. - No error if not found.
remove (item)	>	• Removes item. • Error if not found.
pop ()	>=	Removes & returns a random item.
isdisjoint (set2)		True if sets have no common elements.
issubset (set2)	<=	True if all elements of s1 are in s2.
issuperset (set2)	<	True if all elements of s2 are in s1.

```

# Initial sets
set1 = {1, 2, 3}
set2 = {3, 4, 5}

# 1. add(item)
set1.add(7)
print(set1) # {1, 2, 3, 7}
set1.add((7))
print(set1) # {1, 2, 3, 7}
set1.add('7')
print(set1) # {1, 2, 3, 7, '7'}
print(set1.add([7])) # TypeError: unhashable type: 'list'

print(set1.add(7)) # None
print(set1.add((7))) # None
print(set1.add('7')) # None
print(set1.add([7])) # TypeError: unhashable type: 'list'

# 2. union(set2)
set1.union(set2) # {1, 2, 3, 4, 5, 7}

# 3. update(set2)
temp = set1.copy()
temp.update(set2) # {1, 2, 3, 4, 5, 7}
print(set1) # {1, 2, 3, 7}

# 4. clear()
temp = set1.copy()
temp.clear() # set()

# 5. copy()
copy_set = set1.copy() # {1, 2, 3, 7}

# 6. intersection(set2)
set1.intersection(set2) # {3}

# 7. intersection_update(set2)
temp = set1.copy()
temp.intersection_update(set2) # {3}

# 8. difference(set2)
set1.difference(set2) # {1, 2, 7}

# 9. difference_update(set2)
temp = set1.copy()
temp.difference_update(set2) # {1, 2, 7}

```

```
# 10. symmetric_difference(set2)
set1.symmetric_difference(set2) # {1, 2, 4, 5, 7}

# 11. symmetric_difference_update(set2)
temp = set1.copy()
temp.symmetric_difference_update(set2) # {1, 2, 4, 5, 7}

# 12. discard(item)
set1.discard(2) # {1, 3, 7}

# 13. remove(item)
set1.remove(6) # KeyError if uncommented
# 14. pop()
popped_item = set1.pop()
print(popped_item, set1) # 1 {3, 7}

# 15. isdisjoint(set2)
set1.isdisjoint(set2) # False

# 16. issubset(set2)
{3}.issubset(set2) # True

# 17. issuperset(set2)
set1.issuperset({3}) # True
```

(4) String

- In Python, all string methods return a new string because strings are immutable & cannot be changed in place.
- String methods return a string; some return int, bool, or list.

Methods	Description
<code>str1 + str2</code>	Concatenates two strings.
<code>str * n</code>	Repeats string 'n' times.
<code>len(str)</code>	Returns the no. of characters.
<code>str.upper()</code>	Converts to uppercase.
<code>str.lower()</code>	Converts to lowercase
<code>str.title()</code>	Capitalizes first letter of each word
<code>str.capitalize()</code>	Capitalizes first letter of the string
<code>str.strip()</code>	Removes leading/trailing spaces
<code>str.lstrip()</code>	Removes leading spaces
<code>str.rstrip()</code>	Removes trailing spaces
<code>str.replace(a, b)</code>	Replaces substring a with b.
<code>str.find(sub)</code>	• Index of first occurrence of sub-string. • -1 if not found
<code>str.index(sub)</code>	• Index of first occurrence of sub-string. • Error if not found
<code>str.count(sub)</code>	Counts how many times substring occurs
<code>str.startswith(sub)</code>	True - string starts with substring
<code>str.endswith(sub)</code>	True - string ends with substring
<code>str.isalpha()</code>	True - all characters are letters
<code>str.isdigit()</code>	True - all characters are digits
<code>str.isalnum()</code>	True - all characters are letters/digits
<code>str.isspace()</code>	True - all characters are whitespace
<code>str.islower()</code>	True - all letters are lowercase
<code>str.isupper()</code>	True - all letters are uppercase
<code>str.split()</code>	Splits string into list of words
<code>'sep'.join(list)</code>	Joins list into a string using separator
<code>str[::-1]</code>	Reverses the string (slicing)

```

s1 = "Hello"
s2 = "World"
s3 = "hello world"

s = "  Hello World123  "

print(s1 + s2)           # 'HelloWorld'
print(s1 * 3)            # 'HelloHelloHello'
print(len(s))            # 18
print(s.upper())         # '  HELLO WORLD123  '
print(s.lower())         # '  hello world123  '
print(s.title())         # '  Hello World123  '
print(s.capitalize())    # '  hello world123  '
print(s3.capitalize())   # 'Hello world'
print(s.strip())         # 'Hello World123'
print(s.lstrip())        # 'Hello World123  '
print(s.rstrip())        # '  Hello World123'
print(s.replace("World", "Python")) # '  Hello Python123  '
print(s.find("World"))   # 8
print(s.index("World"))  # 8
print(s.count("l"))      # 3
print(s.startswith("  He")) # True
print(s.endswith("123  ")) # True
print(s.isalpha())       # False
print(s.isdigit())       # False
print(s.isalnum())       # False
print(s.isspace())       # False
print(s.islower())       # False
print(s.isupper())       # False
print(s.split())          # ['Hello', 'World123']
print("--".join(s.split())) # 'Hello--World123'
print(s[::-1])           # '  321dlroW olleH  '

```

(5) Dictionary

In Python, dictionary methods like dict.keys(), dict.values() & dict.items() return **view objects**, not lists; to save memory, reflect real-time changes & allow set operations.

Methods	Description
d.keys() list(d.keys())	• Returns a view object of all the keys. • Returns a list of all keys.
d.values() list(d.values())	• Returns a view object of all the values. • Returns a list of all values.
d.items() list(d.items()) tuple(list(d.items()))	• Returns a view object of all the key-value pairs. • Returns a list of key-value pairs as tuples. • Returns the given list of key-value pairs into a tuple of tuples.
d.get('key')	• Returns value for key • Else None
d.pop('key')	Removes key & returns its value
d.popitem()	Removes & returns the last inserted key-value pair as a tuple .
d.update(other_dict)	Updates dict with key-value pairs from other_dict
d.clear()	Removes all items
d.copy()	Returns shallow copy of dictionary
d.setdefault('key', value)	• Returns value if key exists • Else inserts key with value
key in dict	Checks if key exists
d.fromkeys(iterable) d.fromkeys(iterable, value=None)	• Creates a new dictionary with all keys from the list 'keys' & sets their values to 'None' (default). • Creates a new dictionary with all keys from the list 'keys' & sets all values to '10'.
len(dict)	Returns no. of key-value pairs

```
d = {'a': 1, 'b': 2, 'c': 3}
new_data = {'d': 4}
keys = ['x', 'y', 'z']

# 1. keys()
print(d.keys())          # dict_keys(['a', 'b', 'c'])
print(list(d.keys()))    # ['a', 'b', 'c']

# 2. values()
print(d.values())        # dict_values([1, 2, 3])
print(list(d.values()))  # [1, 2, 3]

# 3. items()
print(d.items())         # dict_items([('a', 1), ('b', 2), ('c', 3)])
print(list(d.items()))   # [('a', 1), ('b', 2), ('c', 3)]
print(tuple(list(d.items()))) # (('a', 1), ('b', 2), ('c', 3))

# 4. get()
print(d.get('b'))        # 2
print(d.get('x'))        # None

# 5. pop()
print(d.pop('c'))        # 3

# 6. popitem()
print(d.popitem())       # ('b', 2)

# 7. update()
d.update(new_data)
print(d)                 # {'a': 1, 'd': 4}

# 8. copy()
print(d.copy())          # {'a': 1, 'd': 4}

# 9. setdefault()
print(d.setdefault('a', 100)) # 1
print(d.setdefault('z', 999)) # 999
print(d)                 # {'a': 1, 'd': 4, 'z': 999}

# 10. 'key' in dict
print('a' in d)          # True
print('x' in d)          # False

# 11. len()
print(len(d))            # 3
```

```
# 12. fromkeys()
d2 = dict.fromkeys(keys)
print(d2)                # {'x': None, 'y': None, 'z': None} → default
                           value is None

d3 = dict.fromkeys(keys, 10)
print(d3)                # {'x': 10, 'y': 10, 'z': 10} → default value is
                           10

# 13. clear()
d.clear()
print(d)                  # {}
```

```

tournament_data = {
    "TeamA": {
        "P101": {"name": "Rahul", "scores": [45, 60, 72, 55], "role":
"Batsman"},
        "P102": {"name": "Vikram", "scores": [10, 25, 30, 18], "role": "All-
rounder"},
        "P103": {"name": "Arjun", "scores": [78, 88, 91, 84], "role":
"Batsman"},
    },
    "TeamB": {
        "P201": {"name": "Rohan", "scores": [50, 43, 70, 65], "role":
"Batsman"},
        "P202": {"name": "Kiran", "scores": [25, 35, 15, 40], "role":
"Bowler"},
        "P203": {"name": "Sanjay", "scores": [60, 90, 85, 75], "role": "All-
rounder"},
    },
    "TeamC": {
        "P301": {"name": "Manoj", "scores": [32, 50, 28, 65], "role":
"Batsman"},
        "P302": {"name": "Deepak", "scores": [5, 12, 20, 10], "role":
"Bowler"},
        "P303": {"name": "Karthik", "scores": [90, 100, 85, 110], "role":
"Batsman"},
    }
}

```

(a) For each employee, calculate their average project rating and add it as a new key "avg_rating" inside their record.

```

for team, players in tournament_data.items():
    for pid, info in players.items():
        total = sum(info["scores"])
        count = len(info["scores"])
        info["avg_score"] = round(total / count, 2)

```

(b) Identify the best performer in each team (player with highest avg_score) and display their name and team.

```

for team, players in tournament_data.items():
    best_player_name = None
    best_avg = -1
    for pid, info in players.items():
        if info["avg_score"] > best_avg:
            best_avg = info["avg_score"]
            best_player_name = info["name"]
    print(team, ":", best_player_name, "-", best_avg)

```

```
# (c) Determine the player with the overall highest average score across all teams and display their details.
```

```
overall_best_name = None
overall_best_team = None
overall_best_score = -1
```

```
for team, players in tournament_data.items():
    for pid, info in players.items():
        if info["avg_score"] > overall_best_score:
            overall_best_score = info["avg_score"]
            overall_best_name = info["name"]
            overall_best_team = team
```

```
print("Name:", overall_best_name)
print("Team:", overall_best_team)
print("Average Score:", overall_best_score)
print()
```

```
# (d) Compute the team performance score for each team as the average of all its players' average scores, and display which team performed the best.
```

```
team_avg_scores = {}
```

```
for team, players in tournament_data.items():
    total_avg = 0
    player_count = 0

    for pid, info in players.items():
        total_avg += info["avg_score"]
        player_count += 1

    team_avg = round(total_avg / player_count, 2)
    team_avg_scores[team] = team_avg
```

```
for t, v in team_avg_scores.items():
    print(t, ":", v)
```

```
# Find Best Team
```

```
best_team_name = None
best_team_score = -1
```

```
for t, v in team_avg_scores.items():
    if v > best_team_score:
        best_team_score = v
        best_team_name = t
```

```
print("Best Team:", best_team_name, "-", best_team_score)
```

```
# (e) Display the names of players whose average score is below their team's  
average – these are underperformers.
```

```
for team, players in tournament_data.items():  
  
    team_average = team_avg_scores[team]  
  
    for pid, info in players.items():  
        if info["avg_score"] < team_average:  
            print(info["name"], "from", team)
```

```

company_data = {
    "E101": {"name": "Anita", "dept": "IT", "projects": {"P1": 8, "P2": 9,
"P3": 7}},
    "E102": {"name": "Ravi", "dept": "Finance", "projects": {"P1": 6, "P3": 8,
"P4": 9}},
    "E103": {"name": "Kiran", "dept": "IT", "projects": {"P2": 7, "P3": 6,
"P4": 8}},
    "E104": {"name": "Meena", "dept": "HR", "projects": {"P1": 9, "P2": 8,
"P3": 8}},
    "E105": {"name": "Divya", "dept": "Finance", "projects": {"P1": 10, "P3":
9, "P4": 10}},
    "E106": {"name": "Ajay", "dept": "HR", "projects": {"P2": 5, "P3": 6,
"P4": 5}}
}

# (a) For each employee, calculate their average project rating and add it as
a new key "avg_rating" inside their record.

for eid, info in company_data.items():
    scores = list(info["projects"].values())
    info["avg_rating"] = round(sum(scores) / len(scores), 2)

# (b) Determine the best-performing employee (highest avg_rating) and display
their name, department, and average rating.

best_name = None
best_dept = None
best_avg = -1

for eid, info in company_data.items():
    if info["avg_rating"] > best_avg:
        best_avg = info["avg_rating"]
        best_name = info["name"]
        best_dept = info["dept"]

print("Name:", best_name)
print("Department:", best_dept)
print("Average Rating:", best_avg)
# Name: Divya
# Department: Finance
# Average Rating: 9.67

```

```

# (c) For each department, calculate and display the average performance rating
# (i.e., average of all employees' avg_rating within that department).
dept_totals = {}
dept_counts = {}

for eid, info in company_data.items():
    d = info["dept"]

    if d not in dept_totals:
        dept_totals[d] = 0
        dept_counts[d] = 0

    dept_totals[d] += info["avg_rating"]
    dept_counts[d] += 1

for d in dept_totals:
    avg = round(dept_totals[d] / dept_counts[d], 2)
    print(d, ":", avg)

# IT : 7.0
# Finance : 8.83
# HR : 6.33

# (d) Identify and display which project (P1-P4) has the highest overall
# average rating across all employees.
project_totals = {}
project_counts = {}
for eid, info in company_data.items():
    for proj, score in info["projects"].items():

        if proj not in project_totals:
            project_totals[proj] = 0
            project_counts[proj] = 0

        project_totals[proj] += score
        project_counts[proj] += 1

best_proj = None
best_proj_avg = -1

for proj in project_totals:
    avg = project_totals[proj] / project_counts[proj]
    if avg > best_proj_avg:
        best_proj_avg = avg
        best_proj = proj

print("Project:", best_proj) # Project: P4
print("Average Score:", round(best_proj_avg, 2)) # Average Score: 8.67

```

```
# (e) Display the names of employees who have scored below their department's  
average rating (these are underperformers).
```

```
for eid, info in company_data.items():  
    dept = info["dept"]  
    dept_avg = dept_totals[dept] / dept_counts[dept]
```

```
    if info["avg_rating"] < dept_avg:  
        print(info["name"], "from", dept)
```

```
# Anita from IT
```

```
# Meena from HR
```

```
# Ajay from HR
```

```
# Ravi from Finance (depends on final department average)
```

Mutable Object	Immutable Objects
In-place modification (list [0] = 5) is allowed.	In-place modification (tuple [0] = 5) is not allowed — gives an error.
Reassignment of the whole variable is allowed — original object gets modified.	Reassignment of the whole variable is allowed — but it creates a new object.
list, dict, set	tuple, str, int, float, bool

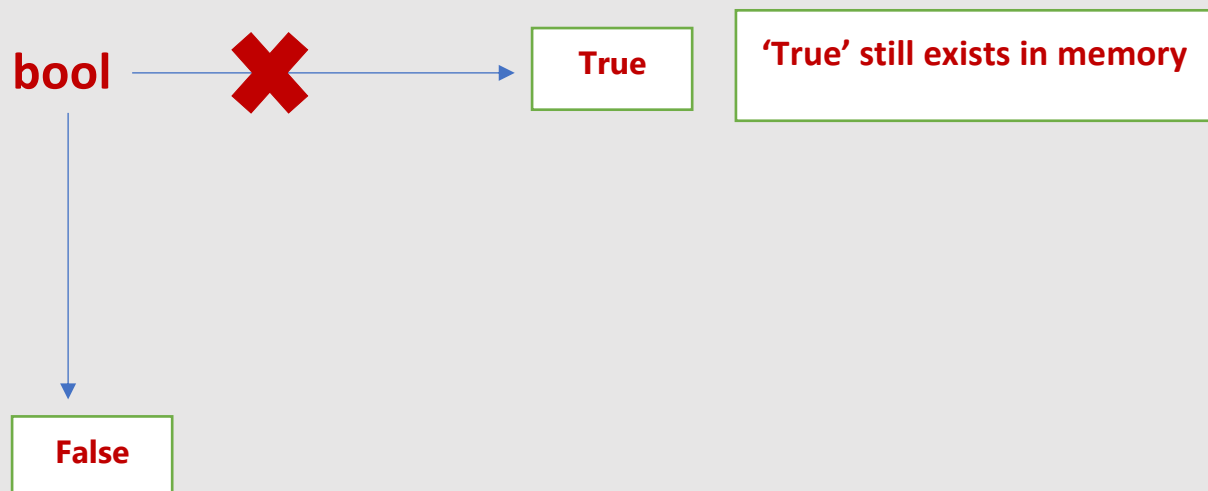
In Place Modification

- Changing a part (like an element — list [0] = 5) inside the same/original object.
- This is not the same as reassigning the whole variable to a new object.

```
bool = True
bool  —————>  True (object ID: e.g. 1001)

bool = not bool
bool  —————>  False (object ID: e.g. 1002)

(True still exists in memory, but bool no longer points to it)
```



```
# ----- Mutable Data Structures -----
# list
list = [1, 2, 3]
list [0] = 10 # [10, 2, 3]

# dictionary
dict = {'a': 1}
dict ['a'] = 100 # {'a': 100}

# set
set = {1, 2, 3}
set.add(4) # {1, 2, 3, 4}

# ----- Immutable Data Structures -----
# int
int = 5
int = int + 1 # 6

# float
float = 3.14
float = float + 1 # 4.14

# bool
bool = True
bool = not bool # False

# str
str = "Anshu"
str = "M" + str [1:] # Mnshu

# tuple
tuple = (1, 2, 3)
# tuple [0] = 1 # TypeError: 'tuple' object does not support item
assignment
print(tuple) # (1, 2, 3)
```

(6) File Handling

- File handling in Python means interacting with files — reading data from them or writing data to them.
- Python provides built-in functions to **create, open, read, write & close** files.

Built-In Functions	Description
file = open ("filename", "mode") as f1	• filename – name or path of the file. • mode – specifies what we want to do with the file (read/write/append, text/binary).
f1. close ()	• Always close files after finishing work. • Use with statement to close automatically.
write ()	- Writes a single string. - Doesn't automatically add a newline character (\n) at the end, so we must include it ourself if we want to write content on separate lines.
writelines ()	- Writes a list of strings. - Doesn't automatically add a newline character (\n) at the end, so we must include it ourself if we want to write content on separate lines.
read ()	- Reads the entire file as a single string. - Keeps newline characters (\n) inside the string.
readline ()	- Reads only the next line (single line) from the file each time it is called. - Returns a string for that one line (includes \n if present). Useful for processing files line-by-line without loading the whole file.
readlines ()	- Reads all lines at once & stores them in a list. - Each list item is one line (includes \n at the end, except possibly the last line).
File Path	String that tells Python where a file/folder is located.
Absolute Path	Full location of the file from root.
Relative Path	Path starting from the current working directory (CWD) — the folder where our script is currently running.

Mode	Description
t	Text mode (default).
b	Binary mode.
r	- Read (default). - File must exist. - Pointer at start.
rb	- Read in binary mode. - File must exist.
r+	- Read & write. - File must exist. - Pointer at start.
rb+	- Read & write in binary. - File must exist.
w	- Write (overwrite). - Creates file if not exists.
wb	- Write in binary (overwrite). - Creates file if not exists.
w+	- Write & read (overwrite). - Creates file if not exists.
wb+	- Write & read in binary (overwrite). - Creates file if not exists.
a	- Append (write at end). - Creates file if not exists.
ab	- Append in binary. - Creates file if not exists.
a+	- Append & read. - Creates file if not exists. - Pointer at end.
ab+	- Append & read in binary. - Creates file if not exists. - Pointer at end.

```

# ----- Absolute Path Example -----
file = open("C:/Users/Anshu/Documents/example.txt", "r")
print(file.read())
file.close()

# ----- Relative Path Example -----
file = open("data/example.txt", "r")
print(file.read())
file.close()

# ----- Using os.path.join for safer path handling -----
import os

# Combine folder name and filename into a complete path safely
folder = "data"
filename = "example.txt"
path = os.path.join(folder, filename) # Joins with correct slash (\ or /
depending on OS)

with open(path, "r") as file:
    print(file.read())

# Example: joining multiple parts of a path
multi_path = os.path.join("home", "user", "documents", "file.txt")
print("Joined multi-part path:", multi_path) # Windows: Joined multi-part
path: home\user\documents\file.txt
# Linux/macOS: Joined multi-part
path: home/user/documents/file.txt

# ----- Checking if a file exists -----
# os.path.exists() returns True if the file/directory exists, else False
if os.path.exists("data/example.txt"):
    print("File exists!")
else:
    print("File not found.")

```

```

# ----- write() -----
with open("example.txt", "w") as f:
    f.write("""Line 1: First Line
Line 2: Second Line
Line 3: Third Line""")

# ----- writelines() -----
with open("example.txt", "a") as f:
    f.writelines(["Line 4: Fourth Line\n", "Line 5: Fifth Line\n", "Line 6:
Sixth Line\n", "Line 7: Seventh Line\n"])
# example.txt
#Line 1: First Line
#Line 2: Second Line
#Line 3: Third LineLine 4: Fourth Line
#Line 5: Fifth Line
#Line 6: Sixth Line
#Line 7: Seventh Line

# ----- read() -----
with open("example.txt", "r") as f:
    print("Read entire file at once:", repr(f.read())) #Read entire file at
once: 'Line 1: First Line\nLine 2: Second Line\nLine 3: Third LineLine 4:
Fourth Line\nLine 5: Fifth Line\nLine 6: Sixth Line\nLine 7: Seventh Line\n'
    print("Read entire file at once:\n", f.read())
#Read entire file at once:
# Line 1: First Line (space is due to comma)
#Line 2: Second Line
#Line 3: Third LineLine 4: Fourth Line
#Line 5: Fifth Line
#Line 6: Sixth Line
#Line 7: Seventh Line

# ----- readline() -----
with open("example.txt", "r") as f:
    print("Read First Line:", repr(f.readline()))
    print("Read Second Line:", repr(f.readline()))
    print("Read Third Line:", repr(f.readline()))
    print("Read Fourth Line:", repr(f.readline()))
    print("Read Fifth Line:", repr(f.readline()))
    print("Read Sixth Line:", repr(f.readline()))
    print("Read Seventh Line:", repr(f.readline()))
#Read First Line: 'Line 1: First Line\n'
#Read Second Line: 'Line 2: Second Line\n'
#Read Third Line: 'Line 3: Third LineLine 4: Fourth Line\n'
#Read Fourth Line: 'Line 5: Fifth Line\n'
#Read Fifth Line: 'Line 6: Sixth Line\n'
#Read Sixth Line: 'Line 7: Seventh Line\n'
#Read Seventh Line: ''

```

```

with open("example.txt", "r") as f:
    print("Read First Line:", f.readline())
    print("Read Second Line:", f.readline())
    print("Read Third Line:", f.readline())
    print("Read Fourth Line:", f.readline())
    print("Read Fifth Line:", f.readline())
    print("Read Sixth Line:", f.readline())
    print("Read Seventh Line:", f.readline())
#Read First Line: Line 1: First Line

#Read Second Line: Line 2: Second Line

#Read Third Line: Line 3: Third LineLine 4: Fourth Line

#Read Fourth Line: Line 5: Fifth Line

#Read Fifth Line: Line 6: Sixth Line

#Read Sixth Line: Line 7: Seventh Line

#Read Seventh Line:

# ----- readlines() -----
with open("example.txt", "r") as f:
    print("Read all line at once:", repr(f.readlines()))
    print("Read all line at once:", repr(f.readlines()))
#Read all line at once: ['Line 1: First Line\n', 'Line 2: Second Line\n',
'Line 3: Third LineLine 4: Fourth Line\n', 'Line 5: Fifth Line\n', 'Line 6:
Sixth Line\n', 'Line 7: Seventh Line\n']
#Read all line at once: []

with open("example.txt", "r") as f:
    print("Read all line at once:", f.readlines())
    print("Read all line at once:", f.readlines())
#Read all line at once: ['Line 1: First Line\n', 'Line 2: Second Line\n',
'Line 3: Third LineLine 4: Fourth Line\n', 'Line 5: Fifth Line\n', 'Line 6:
Sixth Line\n', 'Line 7: Seventh Line\n']
#Read all line at once: []

```

```
# ----- append() -----
with open("example.txt", "a") as f:
    f.write("Appended Line 1: Adding more text.\n")
    f.write("Appended Line 2: Appending continues.\n")
#Line 1: First Line
#Line 2: Second Line
#Line 3: Third LineAppended Line 1: Adding more text.
#Appended Line 2: Appending continues.

with open("example.txt", "r") as f:
    content = f.read()
    print("File content after appending:\n", content)
#File content after appending:
# Line 1: First Line
#Line 2: Second Line
#Line 3: Third LineAppended Line 1: Adding more text.
#Appended Line 2: Appending continues.

# ----- File Handling Functions -----
fo = open("foo.txt", "wb")
print ("Name of the file: ", fo.name) #Name of the file: foo.txt
print ("Closed or not: ", fo.closed) #Closed or not: False
print ("Opening mode: ", fo.mode) #Opening mode: wb
fo.close()
```

(7) Exception Handling

- **Exceptions** are errors that occur **during the execution** of a program.
- Without handling, these errors will crash our program.
- **Exception handling** allows us to **respond to these errors gracefully**.

Common Exceptions

Exception Type	Cause
ZeroDivisionError	Dividing by zero
ValueError	Invalid value for data conversion
FileNotFoundError	File doesn't exist
TypeError	Wrong data type used
IndexError	List index out of range
KeyError	Missing key in dictionary

Syntax of Exception Handling

```
try:
    # Code that might cause an exception
except SomeException:
    # Code that runs if exception occurs
else:
    # Code that runs if NO exception occurs
finally:
    # Code that ALWAYS runs (no matter what)
```

Example

```
# 1. ZeroDivisionError & ValueError
try:
    x = int(input("Enter a number: "))
    print(10 / x)
except ZeroDivisionError: # Error #1
    print("You can't divide by zero!")
except ValueError: # Error #2
    print("Please enter a valid number.")

# 2. Using else block (runs only if no exception)
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except ZeroDivisionError: # Error #1
    print("Can't divide by zero.")
except ValueError: # Error #2
    print("Invalid input.")
else:
    print("Result is:", result)
```

```

# 3. Using finally block (always run), handling FileNotFoundError
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found.")
else:
    print("File content:\n", content)
finally:
    print("Execution complete.")
    #f.close() # Uncomment if file opened successfully

# 4. Catching multiple exceptions in one block
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except (ZeroDivisionError, ValueError):
    print("Error: Invalid number or division by zero.")

# 5. Dictionary lookup with KeyError handling
student = {"name": "Alice", "age": 20}
try:
    print(student["grade"])
except KeyError: # Error #4
    print("Grade not found!")

# 6. IndexError
names = ["Alice", "Bob"]
try:
    print(names[5])
except IndexError:
    print("That index does not exist.")

# 7. Function using try-except-else-finally
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError: # Error #1
        print("Error: Division by zero.")
    else:
        print("Result is:", result)
    finally:
        print("Operation complete.")

divide(10, 0)

# 8. Catch-all generic exception (catches any error, use with care)
try:
    val = int("hello")
except Exception as e:
    print("An error occurred:", e)

```

(8) Mean, Median & Mode

```
# -----1. Without using built-in functions-----
data = [10, 20, 20, 40, 40, 60]

# Mean
mean = sum(data) / len(data)
print(f"Mean: {mean}") # Mean: 31.666666666666668

# Median
array = sorted(data)
n = len(array)
if n % 2 == 1:
    median = array[n // 2]
else:
    mid1 = array[n // 2 - 1]
    mid2 = array[n // 2]
    median = (mid1 + mid2) / 2
print(f"Median: {median}") # Median: 30.0

# Mode
frequency = {}
for num in data:
    frequency[num] = frequency.get(num, 0) + 1
mode = max(frequency, key=frequency.get)
print(f"Mode: {mode}") # Mode: 20

# -----2. Using statistics module-----
import statistics as stats

mean = stats.mean(data)
median = stats.median(data)
mode = stats.mode(data)

print(f"Mean: {mean}") # Mean: 31.666666666666668
print(f"Median: {median}") # Median: 30.0
print(f"Mode: {mode}") # Mode: 20
```

```
# -----3. Using numpy and scipy-----
import numpy as np
from scipy import stats

mean = np.mean(data)
median = np.median(data)
mode_result = stats.mode(data)

print(f"Mean: {mean}")           # Mean: 31.666666666666668
print(f"Median: {median}")       # Median: 30.0
print(f"Mode: {mode_result.mode[0]}") # Mode: 20
print(f"Count: {mode_result.count[0]}") # Count: 2

# -----4. Using pandas-----
import pandas as pd

series = pd.Series(data)

mean = series.mean()
median = series.median()
mode = series.mode()

print(f"Mean: {mean}")           # Mean: 31.666666666666668
print(f"Median: {median}")       # Median: 30.0
print(f"Mode(s): {mode.tolist()}") # Mode(s): [20]
```

Series (in Pandas)

- 1-D labelled array (like a column in Excel or a single list).
- It can hold any data type: integers, floats, strings, etc.

```
import pandas as pd

data = [10, 20, 30, 40]
series = pd.Series(data)
print(series)
```

Output

```
0    10
1    20
2    30
3    40
dtype: int64
```

Data-Frame (in Pandas)

- 2-D labelled data structure with rows & columns (like a spreadsheet or SQL table).
- It's basically a collection of Series.

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Marks": [85, 90, 95]
}
df = pd.DataFrame(data)
print(df)
```

Output

```
   Name  Marks
0  Alice     85
1   Bob     90
2 Charlie     95
```

Feature	Series	Data-Frame
Dimensionality	1D (like a list or single column)	2D (like a table with rows and columns)
Structure	One column with an index	Multiple columns with an index
Data Type	Holds single data type (can be mixed but usually one type)	Each column can have different data types

Example 1

```
import statistics as stats

# Step 1: Define dictionary with student names and their Math marks
student_marks = {
    "Alice": 85,
    "Bob": 90,
    "Charlie": 78,
    "David": 92,
    "Eva": 88
}

# Step 2: Extract only the marks
marks = list(student_marks.values())

# Step 3: Compute statistics
mean = stats.mean(marks)
median = stats.median(marks)
mode = stats.mode(marks)

# Step 4: Display results
print("Student Marks Dictionary:", student_marks)
print("Descriptive Statistics (Math Marks):")
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)

#Student Marks Dictionary: {'Alice': 85, 'Bob': 90, 'Charlie': 78, 'David': 92, 'Eva': 88}
#Descriptive Statistics (Math Marks):
#Mean: 86.6
#Median: 88
#Mode: 85
```

Example 2

```
import pandas as pd

# Step 1: Create a dictionary with student data
data = {
    "Student": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Math": [85, 90, 78, 92, 88],
    "Science": [92, 88, 84, 91, 89],
    "English": [80, 86, 82, 85, 88]
}

# Step 2: Create a DataFrame
df = pd.DataFrame(data)

# Step 3: Display the full DataFrame
print(" Student Marks DataFrame:\n")
print(df)

# Step 4: Accessing a column
print("\n Math Marks:")
print(df["Math"])

# Step 5: Accessing a row (for example, 2nd student i.e. Bob)
print("\n Row 1 - Bob's Record:")
print(df.loc[1])

# Step 6: Descriptive statistics for each subject
print("\n Descriptive Statistics:")

for subject in ["Math", "Science", "English"]:
    print(f"\nSubject: {subject}")
    print("Mean:", df[subject].mean())
    print("Median:", df[subject].median())
    print("Mode:", df[subject].mode())

import pandas as pd

# Step 1: Create a dictionary with student data
data = {
    "Student": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Math": [85, 90, 78, 92, 88],
    "Science": [92, 88, 84, 91, 89],
    "English": [80, 86, 82, 85, 88]
}

# Step 2: Create a DataFrame
df = pd.DataFrame(data)
```

```

# Step 3: Display the full DataFrame
print(" Student Marks DataFrame:\n")
print(df)

# Step 4: Accessing a column
print("\n Math Marks:")
print(df["Math"])

# Step 5: Accessing a row (for example, 2nd student i.e. Bob)
print("\n Row 1 - Bob's Record:")
print(df.loc[1])

# Step 6: Descriptive statistics for each subject
print("\n Descriptive Statistics:")

for subject in ["Math", "Science", "English"]:
    print(f"\nSubject: {subject}")
    print("Mean:", df[subject].mean())
    print("Median:", df[subject].median())
    print("Mode:", df[subject].mode())

# Student Marks DataFrame:

#   Student  Math  Science  English
# 0   Alice    85      92      80
# 1    Bob     90      88      86
# 2  Charlie    78      84      82
# 3   David    92      91      85
# 4    Eva     88      89      88

# Math Marks:
# 0    85
# 1    90
# 2    78
# 3    92
# 4    88
# Name: Math, dtype: int64

# Row 1 - Bob's Record:
# Student      Bob
# Math          90
# Science       88
# English       86
# Name: 1, dtype: object

```

```
# Descriptive Statistics:

# Subject: Math
# Mean: 86.6
# Median: 88.0
# Mode: 0      78
# 1      85
# 2      88
# 3      90
# 4      92
# Name: Math, dtype: int64

# Subject: Science
# Mean: 88.8
# Median: 89.0
# Mode: 0      84
# 1      88
# 2      89
# 3      91
# 4      92
# Name: Science, dtype: int64

# Subject: English
# Mean: 84.2
# Median: 85.0
# Mode: 0      80
# 1      82
# 2      85
# 3      86
# 4      88
# Name: English, dtype: int64
```

Example 3

```
import numpy as np
from scipy import stats

# Step 1: Define dictionary with student names and their marks
student_marks = {
    "Alice": 85,
    "Bob": 90,
    "Charlie": 78,
    "David": 92,
    "Eva": 88
}

# Step 2: Convert marks to a NumPy array
marks_array = np.array(list(student_marks.values()))

# Step 3: Compute statistics
mean = np.mean(marks_array)
median = np.median(marks_array)
mode_result = stats.mode(marks_array)

# Step 4: Display results
print("Student Marks Dictionary:", student_marks)
print("\nDescriptive Statistics (Math Marks):")
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode_result.mode[0])

# Student Marks Dictionary: {'Alice': 85, 'Bob': 90, 'Charlie': 78, 'David': 92, 'Eva': 88}

# Descriptive Statistics (Math Marks):
# Mean: 86.6
# Median: 88.0
# Mode: 78
```

Example 4

```
# You are provided with the following dataset of students including their
names, ages, and weekly working hours:
# data = {
#     "Name": ["Alice", "Bob", "Charlie", "David", "Emma", "Frank", "Grace"],
#     "Age": [21, 22, 23, 21, 22, 24, 23],
#     "Working_Hours": [40, 35, 45, 35, 40, 50, 40]
# }
# Tasks:
# 1. Create a DataFrame from the above dictionary.
# 2. Display the complete DataFrame.
# 3. Calculate and print the mean, median, and mode of the Working_Hours
column.

import pandas as pd
import statistics as stats

# Step 1: Create the DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Emma", "Frank", "Grace"],
    "Age": [21, 22, 23, 21, 22, 24, 23],
    "Working_Hours": [40, 35, 45, 35, 40, 50, 40]
}

df = pd.DataFrame(data)

# Step 2: Display the complete DataFrame
print("Complete DataFrame:\n")
print(df)

# Step 3: Calculate statistics for Working_Hours column
working_hours = df["Working_Hours"]

mean_hours = working_hours.mean()
median_hours = working_hours.median()
mode_hours = stats.mode(working_hours)

print("\nDescriptive Statistics for Working_Hours:")
print(f"Mean: {mean_hours}")
print(f"Median: {median_hours}")
print(f"Mode: {mode_hours}")
```

```
# Complete DataFrame:
#      Name  Age  Working_Hours
# 0    Alice   21           40
# 1     Bob    22           35
# 2  Charlie   23           45
# 3    David   21           35
# 4     Emma   22           40
# 5    Frank   24           50
# 6    Grace   23           40

# Descriptive Statistics for Working_Hours:
# Mean: 40.714285714285715
# Median: 40.0
# Mode: 40
```

Example 5

```
# -----VARIANCE & STANDARD DEVIATION-----
data = [10, 20, 20, 40, 40, 60]

# Without built-in
mean = sum(data) / len(data)
variance = sum((x - mean) ** 2 for x in data) / len(data)
std_dev = variance ** 0.5
print(f"Variance : {variance}")
print(f"Standard Deviation : {std_dev}")

# Using statistics module
import statistics as stats
print("Variance :", stats.pvariance(data)) # population variance
print("Standard Deviation: ", stats.pstdev(data)) # population std dev

# Using NumPy
import numpy as np
print("Variance: ", np.var(data))
print("Standard Deviation: ", np.std(data))

# Using Pandas
import pandas as pd
data = [10, 20, 20, 40, 40, 60]
series = pd.Series(data)

variance = series.var(ddof=0) # ddof=0 → population variance, ddof=1 → sample
variance
print("Variance: ", variance)

std_dev = series.std(ddof=0) # ddof=0 → population std dev
print("Standard Deviation: ", std_dev)
```

(9) Class & Object

```
# Example 1
class MyClass:
    x = 5

    def show_message(self):
        # self refers to the current object calling this method.
        # Python automatically passes the object (p1) as the first argument to
        the method.
        # If we remove self --> TypeError: MyClass.show_message () takes 0
        positional arguments but 1 was given.
        print ("Hello from MyClass!")

p1 = MyClass()
print ("Value of x:", p1.x) # Value of x: 5
p1.show_message() # Hello from MyClass!

# Example 2
class Person:
    # __init__ is a special built-in method in Python classes.
    # Commonly used to initialize data members (set up the object's values).
    def __init__(self, name, age): # __init__ runs automatically on object
creation
        self.name = name          # store the given name in the object
        self.age = age           # store the given age in the object

p1 = Person("John", 36)

# Access object properties
print(p1.name) # John
print(p1.age)  # 36

# Example 3
class Person:
    def __init__(mysillyobject, name, age): # Using mysillyobject instead of
self
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc): # Using abc instead of self
        print("Hello my name is " + abc.name)

# Create object
```

```
p1 = Person("John", 36)
p1.myfunc() # Hello my name is John

# Modify object property
p1.age = 40
print("Updated age:", p1.age) # Updated age: 40

# Delete object property
del p1.age
# print(p1.age) # ✗ Cause an AttributeError if uncommented

# Delete object
del p1
# print(p1) # ✗ Cause a NameError if uncommented

# Empty class using pass
class EmptyClass:
    pass
```

(10) Recursive Function

A **recursive function** is a function that **calls itself** in order to solve a problem.

Syntax

```
def function-name(parameters):
```

```
    if base-condition:
```

```
        return base-result
```

```
    else:
```

```
        return function-name (modified-parameters)
```

```
# Factorial of n
def factorial(n):
    if n == 0:
        return 1 # Base case
    else:
        return n * factorial(n - 1) # Recursive call

print(factorial(5)) # Output: 120

# factorial(5) = 5 * factorial(4)
# factorial(4) = 4 * factorial(3)
# factorial(3) = 3 * factorial(2)
# factorial(2) = 2 * factorial(1)
# factorial(1) = 1 * factorial(0)
# factorial(0) = 1 (base case)
# Then resolve backwards:
# factorial(1) = 1 * 1 = 1
# factorial(2) = 2 * 1 = 2
# factorial(3) = 3 * 2 = 6
# factorial(4) = 4 * 6 = 24
# factorial(5) = 5 * 24 = 120

# sum of first n natural numbers
def sum_natural(n):
    if n == 1:
        return 1 # Base case
    else:
        return n + sum_natural(n - 1) # Recursive call

print(sum_natural(5)) # Output: 15
```

(11) Lambda Function

What is a Lambda Function?

- It's an **anonymous function** (no name).
- **lambda**: keyword to define the function.
- **arguments**: Takes any no. of arguments (comma-separated input parameters).
- **expression**: Has only one/single expression that is evaluated & returned.

Syntax: lambda arguments: expression

Feature	Lambda Function	Regular Function (def)
Definition	Single expression using lambda.	Multiple lines of code using def.
Name	Anonymous (unless assigned to a variable).	Must have a name.
Statements	Only a single expression allowed.	Can have multiple statements and lines.
Documentation	Cannot have a docstring.	Can include docstrings.
Reusability	Best for short, simple, temporary functions.	Suitable for reusable, complex logic.

```
# 1. Convert a string to uppercase using lambda and upper()
# Explanation: s2 is a lambda function that takes a string 'func' and returns
func.upper()
s1 = 'welcome'
s2 = lambda func: func.upper()
print(s2(s1)) # WELCOME

# 2. Lambda with if-else
check = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check(4)) # Even
print(check(7)) # Odd

# 3. Lambda with nested-if-else
n = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
# n = lambda x: ("Positive" if x > 0 else ("Negative" if x < 0 else "Zero"))
print(n(5)) # Positive
print(n(-3)) # Negative
print(n(0)) # Zero
```

```

# 4. Lambda vs def function example: square of a number
sq = lambda x: x ** 2
def sqdef(x):
    return x ** 2

print(sq(3)) # 9
print(sqdef(3)) # 9

# 5. Lambda with multiple results (tuple)
# calc returns a tuple containing the sum and product of x and y
calc = lambda x, y: (x + y, x * y)
res = calc(3, 4)
print(res) # (7, 12)

# 6. Lambda with filter (function, iterable) --> picks elements
# --> Selects only those elements from the iterable for which the function
# returns True.
# Returns: An iterator with the filtered elements.
# Filter even numbers from a list
num = [1, 2, 3, 4, 5, 6]
even_nums = filter(lambda x: x % 2 == 0, num)
print(list(even_nums)) # [2, 4, 6]

# 7. Lambda with map (function, iterable) --> changes elements.
# --> Applies the function to every element in the iterable and returns the
# results.
# Returns: An iterator with the transformed elements.
# Double each number in a list
num = [1, 2, 3, 4]
doubled = map(lambda x: x * 2, num)
print(list(doubled)) # [2, 4, 6, 8]

# 8. Lambda with reduce () --> reduce () "folds" the list into a single value
# by applying the lambda function repeatedly on pairs of items.
# Find product of all numbers in a list
from functools import reduce
num = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, num)
print(product) # 24 {[1*2)*3]*4}

```

(12) Keywords

```
# 1. Positional Parameters
def greet_positional(name, age):
    print(f"Hello {name}, you are {age} years old.")

greet_positional("Alice", 25) # Hello Alice, you are 25 years old.
# greet_positional(25, "Alice") # Logical Error: arguments swapped but still
valid syntax

# -----

# 2. Keyword (Named) Parameters
def greet_keyword(name, age):
    print(f"Hello {name}, you are {age} years old.")

greet_keyword(age=30, name="Bob") # Correct: order doesn't matter
# greet_keyword(name="Bob") # ✗ Missing required argument 'age'
# greet_keyword(age=30, nam="Bob") # ✗ Typo in keyword argument 'nam'
(should be 'name')

# -----

# 3. Default Parameters
def greet_default(name="Guest", age=18):
    print(f"Hello {name}, you are {age} years old.")

greet_default() # Uses defaults
greet_default("Charlie") # Override name, age default used
greet_default(age=40) # Override age, name default used
# greet_default(, 40) # ✗ Syntax error: can't skip positional without
keyword

# -----

# 4. Variable-length Positional Arguments (*args)
def add_numbers(*args):
    total = sum(args)
    print(f"Sum is: {total}")

add_numbers(1, 2)
add_numbers(3, 4, 5)
add_numbers() # Valid, sum is 0
# add_numbers(1, a=2) # ✗ Error: unexpected keyword argument 'a'

# -----
```

```

# 5. Variable-length Keyword Arguments (**kwargs)
def print_info(**kwargs):
    print("Info:")
    for key, value in kwargs.items():
        print(f"{key} = {value}")

print_info(name="Alice", age=25, city="NY")
print_info() # Valid: no arguments passed
# print_info("Alice", age=25) # ✗ Error: positional argument passed to
**kwargs

# -----

# 6. Keyword-only Arguments (after *, must be named)
def keyword_only(a, b, *, c):
    print(f"a={a}, b={b}, c={c}")

keyword_only(1, 2, c=3) # Correct
# keyword_only(1, 2, 3) # ✗ Error: c must be passed as keyword argument
# keyword_only(a=1, b=2, c=3) # Valid, because a and b are positional or
keyword

# -----

# 7. Positional-only Arguments (before /, must be positional) - Python 3.8+
def positional_only(a, b, /, c):
    print(f"a={a}, b={b}, c={c}")

positional_only(1, 2, c=3) # Correct
# positional_only(a=1, b=2, c=3) # ✗ Error: a and b must be positional
# positional_only(1, 2, 3) # Valid (c passed positionally as well)

```

(13) List & Dictionary Comprehensions

```
# -----LIST COMPREHENSIONS-----

# Syntax: [expression for item in iterable if condition]
# Creates a new list in a single line from an existing iterable.

# Example 1: Square of numbers 1 to 5
squares = [x**2 for x in range(1, 6)]
print("Squares:", squares) # [1, 4, 9, 16, 25]

# Example 2: Even numbers from 1 to 10
evens = [x for x in range(1, 11) if x % 2 == 0]
print("Evens:", evens) # [2, 4, 6, 8, 10]

# Example 3: Convert all words to uppercase
words = ["apple", "banana", "cherry"]
upper_words = [w.upper() for w in words]
print("Uppercase:", upper_words) # ['APPLE', 'BANANA', 'CHERRY']

# Nested list comprehension: Flatten a 2D list
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
print("Flattened list:", flat) # [1, 2, 3, 4, 5, 6]

# -----DICTIONARY COMPREHENSIONS-----

# Syntax: {key_expr: value_expr for item in iterable if condition}

# Example 1: Map numbers to their squares
square_dict = {x: x**2 for x in range(1, 6)}
print("Square dict:", square_dict) # {1:1, 2:4, 3:9, 4:16, 5:25}

# Example 2: Filter items with value > 2
original = {"a": 1, "b": 3, "c": 2}
filtered = {k: v for k, v in original.items() if v > 2}
print("Filtered dict:", filtered) # {'b': 3}

# Example 3: Invert a dictionary (swap keys and values)
inverted = {v: k for k, v in original.items()}
print("Inverted dict:", inverted) # {1: 'a', 3: 'b', 2: 'c'}
```

(14) Decorators

A decorator is a function that takes another function as input and adds extra functionality to it without modifying the original code & returns the modified function.

Think of it like wrapping a gift box – the box (original function) is still there, but now it has shiny wrapping paper (extra features).

```
def my_decorator(func):
    def wrapper():
        print("Before function runs...")
        func()
        print("After function runs...")
    return wrapper
```

```
@my_decorator
def say_hello():
    print("Hello, World!")
```

```
say_hello()
# Output:
# Before function runs...
# Hello, World!
# After function runs...
```

```
# Example: Decorator with arguments
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
            return wrapper
        return decorator
```

```
@repeat(3)
def greet(name):
    print(f"Hello {name}!")
```

```
greet("Anshu") # Runs 3 times
```

```
#Before function runs...
#Hello, World!
#After function runs...
#Hello Anshu!
#Hello Anshu!
#Hello Anshu!
```

```

# -----BASIC DECORATOR EXAMPLE-----

def make_pretty(func):
    # inner function adds extra behavior
    def inner():
        print("I got decorated")    # This runs when inner() is called
        func()                    # Calls the original ordinary()
    return inner                  # Return inner, replacing the original
function

@make_pretty
def ordinary():
    # Original function
    print("I am ordinary")        # Runs when func() inside inner() is called

print("1. Basic Decorator Example:")

# Call ordinary(), but it now points to inner()
ordinary()

# -----DECORATOR WITH PARAMETERS-----

def smart_divide(func):
    # inner function handles parameters a and b
    def inner(a, b):
        print(f"I am going to divide {a} and {b}") # Extra behavior before
original function
        if b == 0:
            print("Whoops! cannot divide")          # Prevent division by zero
            return
        return func(a, b) # Call the original divide function
    return inner      # Return inner, replacing original function

@smart_divide
def divide(a, b):    # Original divide function
    print(a / b)    # This runs if b is not zero

print("\n2. Decorator with Parameters Example:")

divide(2, 5) # Call divide()
divide(2, 0) # Call divide()

```

```

# -----CHAINING MULTIPLE DECORATORS-----

def star(func):
    # Outer decorator that prints stars before and after
    def inner(*args, **kwargs):
        print("*" * 15)          # Before calling function
        func(*args, **kwargs)    # Call the wrapped function
        print("*" * 15)          # After calling function
    return inner

def percent(func):
    # Inner decorator that prints % before and after
    def inner(*args, **kwargs):
        print("%" * 15)          # Before calling function
        func(*args, **kwargs)    # Call the wrapped function
        print("%" * 15)          # After calling function
    return inner

# Applying multiple decorators
@star      # outer decorator (runs last)
@percent   # inner decorator (runs first)
def printer(msg): # Original function
    print(msg)

print("\n3. Chaining Decorators Example:")

# Calling printer(), but it is wrapped first by percent, then star
printer("Hello")

#1. Basic Decorator Example:
#I got decorated
#I am ordinary

#2. Decorator with Parameters Example:
#I am going to divide 2 and 5
#0.4
#I am going to divide 2 and 0
#Whoops! cannot divide

#3. Chaining Decorators Example:
#*****
#%%%%%%%%%%
#Hello
#%%%%%%%%%%
#*****

```

(15) Module, Package & Library

(a) Module

- A **single .py** file that may contain code such as functions, classes, variables or even be empty.
- Can be reused by **importing** it.

Module	Description
math	Math functions (sqrt, factorial, pi, etc.)
os	Interact with the operating system
sys	Access system-specific parameters
random	Generate random numbers
datetime	Handle dates and times

(b) Package

- A **folder (directory)** that contains one or more modules (or sub-packages) & **must include** an **`__init__.py`** file (even if empty) to indicate it is a package.
- Can be reused by **importing modules** inside it.

(c) Library

- A **folder (directory)** that contains a collection of modules & packages.
- Can be reused by **importing modules** inside a package or library.

Note

- **import** → module name
- **from** → folder (package/library) name

```
LibraryRoot/          → Library (top-level container)
|
├─ mymodule.py        → Module
|
├─ mypackage1/        → Package
|   ├─ math_utils.py  → Module
|   ├─ string_utils.py → Module
|   └─ mypackage2/    → Sub-package
|       ├─ submod1.py → Module
|       └─ submod2.py → Module
|
└─ mylibrary/         → Package (or Sub-library)
    ├─ operations.py  → Module
    └─ textops.py     → Module
```

```

# Create a Custom Module
# my_module.py
def add(a, b):
    return a + b
def greet(name):
    return f"Hello, {name}!"

# main.py (Import & Use Custom Module)
import my_module
print(my_module.add(5, 3)) # 8
print(my_module.greet("Alice")) # Hello, Alice!

# --- Import built-in modules (math, os, sys, random, datetime) ---
import math
print(math.sqrt(16)) # 4.0
print(math.pi) # 3.141592653589793
print(math.factorial(5)) # 120
print(math.pow(2, 3)) # 8.0
print(math.gcd(24, 36)) # 12
print(math.ceil(4.2)) # 5
print(math.floor(4.8)) # 4

import os
print(os.getcwd()) # Current working directory --> e.g., I:\My Drive\Fall Semester\Python\Python
print(os.name) # OS name --> 'nt' for Windows, 'posix' for Linux/Mac
print(os.listdir()) # List of files in the current directory
os.mkdir("test_dir") # Create new folder named 'test_dir'
os.rmdir("test_dir") # Remove the created folder

import sys
print(sys.version) # Python version --> e.g., 3.12.5
print(sys.platform) # e.g., win32, linux, darwin
print(sys.maxsize) # Maximum integer size --> e.g., 9223372036854775807
print(sys.argv) # Command-line arguments list --> e.g., ['script.py']

import random
random.seed(42) # Same random numbers will come out every time we run the code.
42 has no special mathematical meaning here – we can use any number.
print(random.randint(1, 10)) # Random number 1-10 [a, b] --> 2
print(random.choice(['a', 'b', 'c'])) # Random element --> 'c'
print(random.random()) # Random float between 0 and 1 [a, b) -->
0.6394267984578837
print(random.sample(range(1, 20), 5)) # Random sample of 5 numbers [a, b] -->
[4, 1, 9, 8, 19]
nums = [1, 2, 3, 4, 5]
random.shuffle(nums)
print(nums) # Shuffled list --> [4, 5, 1, 2, 3]

```

```
import datetime
# Get the current date and time
now = datetime.datetime.now()
print("Current Date and Time:", now) # 2025-08-12 23:45:05.935562

# Get only the current date
today = datetime.date.today()
print("Today's Date:", today) # 2025-08-12

# Create a specific date
my_birthday = datetime.date(2002, 8, 29)
print("My Birthday:", my_birthday) # 2002-08-29

# Format date into a readable string
formatted_date = now.strftime("%d-%m-%Y %H:%M:%S")
print("Formatted Date & Time:", formatted_date) # 12-08-2025 23:45:05

# Calculate difference between two dates
future_date = datetime.date(2025, 12, 31)
days_left = future_date - today
print("Days left until 2025 ends:", days_left.days) # 141

# Add days to current date
next_week = today + datetime.timedelta(days=7)
print("Date after 7 days:", next_week) # 2025-08-19

# Subtract days from current date
last_week = today - datetime.timedelta(days=7)
print("Date 7 days ago:", last_week) # 2025-08-05

# Create a time object
t = datetime.time(14, 30, 0)
print("Time object:", t) # 14:30:00

# Replace year in datetime object
new_date = now.replace(year=2026)
print("Date with replaced year:", new_date) # 2026-08-12 23:45:05.935562

# Parse string to datetime object
dt = datetime.datetime.strptime("21-08-2025 15:45", "%d-%m-%Y %H:%M")
print("Parsed datetime:", dt) # 2025-08-21 15:45:00

# ISO format of datetime
print("ISO format:", now.isoformat()) # 2025-08-12T23:45:05.935562
# Day of the week (Monday=0, Sunday=6)
print("Weekday number:", now.weekday()) # 0 for Monday
# ISO day of the week (Monday=1, Sunday=7)
print("ISO Weekday number:", now.isoweekday()) # 1 for Monday
```

NumPy

What is NumPy (Numerical Python)

Library used for numerical calculations to solve complex scientific problems.

Why NumPy?

- Python lists are slower & use more memory for large calculations.
- NumPy arrays (**ndarray**) are faster & more memory-efficient than Python lists.
- Supports vectorized operations, meaning we can perform mathematical operations on entire arrays without using loops.
- Provides powerful functions for linear algebra, statistics & random number generation.

```

import numpy as np

# 1. Creating NumPy Arrays
# 1D array: stores numbers in a single row
print("1D Array:", np.array([1, 2, 3, 4]))      # [1 2 3 4]

# 2D array: stores numbers in rows and columns
print("2D Array:", np.array([[1, 2], [3, 4]]))  # [[1 2]
                                                #          [3 4]]

# Zeros matrix: 3x3 matrix filled with 0's
print("Zeros:", np.zeros((3,3)))                # [[0. 0. 0.]
                                                #          [0. 0. 0.]
                                                #          [0. 0. 0.]]

# Ones matrix: 2x4 matrix filled with 1's
print("Ones:", np.ones((2,4)))                  # [[1. 1. 1. 1.]
                                                #          [1. 1. 1. 1.]]

# Identity matrix: square matrix with 1's on diagonal
print("Identity Matrix:", np.eye(3))             # [[1. 0. 0.]
                                                #          [0. 1. 0.]
                                                #          [0. 0. 1.]]

# Arange: (a, b, step) --> [a, b) with a step value
print("Arange:", np.arange(0, 10, 2))           # [0 2 4 6 8]

# Linspace: (start, stop, num) --> [a, b] generates 'num' numbers evenly
# spaced between 'start' & 'stop'.
print("Linspace:", np.linspace(0,1,5))          # [0.    0.25 0.5   0.75 1.   ]

# 2. Array Attributes --> Provides information about the array
# shape - no. of rows & columns.
# ndim - no. of dimensions (axes --> 1D, 2D, 3D) .
# dtype - data type of elements
arr2 = np.array([[1, 2], [3, 4]])
print("Shape:", arr2.shape)                     # (2, 2)
print("Number of dimensions:", arr2.ndim)        # 2
print("Data type:", arr2.dtype)                 # int64

# 3. Indexing & Slicing
arr = np.array([10, 20, 30, 40, 50])
print("First & last index:", arr[0], arr[-1])   # 10 50
print("Slice 1:4:", arr[1:4])                   # [20 30 40]

arr2d = np.array([[1,2,3],[4,5,6]])
print("Element at row 0, col 1:", arr2d[0,1])   # 2
print("All rows, col 1:", arr2d[:,1])           # [2 5]

```

```

print("Row 1:", arr2d[1,:])           # [4 5 6]

# 4. Array Operations: Perform element-wise arithmetic-mathematical
operations.
a = np.array([1,2,3])
b = np.array([4,5,6])
print("Addition:", a+b)               # [5 7 9]
print("Multiplication:", a*b)        # [4 10 18]
print("Square:", a**2)               # [1 4 9]
print("Sine:", np.sin(a))            # [0.84147098 0.90929743
0.14112001]

# 5. Reshaping & Manipulation (Change the shape, flatten, transpose, or
combine arrays for easier calculations)
# shape - Changes the shape of an array without modifying its data.
# flatten - Converts a multi-dimensional array into a 1D array without
changing the data.
# T - Transpose of a 2D array: flips rows & columns
# concatenate - Combines two/more arrays along an existing axis.
arr = np.arange(1,13)
arr2d = arr.reshape(3,4)
print("Reshaped 3x4:", arr2d)        # [[ 1  2  3  4]
# [ 5  6  7  8]
# [ 9 10 11 12]]

print("Flattened:", arr2d.flatten())  # [
1  2  3  4  5  6  7  8  9 10 11 12]

print("Transposed:", arr2d.T)        # [[ 1  5  9]
# [ 2  6 10]
# [ 3  7 11]
# [ 4  8 12]]

print("Concatenate arrays:", np.concatenate((a,b))) # [1 2 3 4 5 6]

# 6. Statistical Functions (Calculate mean, median, S.D., sum, min & max of
arrays)
arr_stat = np.array([1,2,3,4,5])
print("Mean:", np.mean(arr_stat))    # 3.0
print("Median:", np.median(arr_stat)) # 3.0
print("Standard Deviation:", np.std(arr_stat)) # 1.4142
print("Sum:", np.sum(arr_stat))      # 15
print("Min:", np.min(arr_stat))      # 1
print("Max:", np.max(arr_stat))      # 5

# 7. Boolean Masking & Fancy Indexing Select elements using conditions
(masking) or specific indices (fancy indexing).
mask = arr_stat > 3

```

```

print("Boolean Mask:", mask)                # Output: [False False
False True True]
print("Elements > 3:", arr_stat[mask])      # Output: [4 5]
print("Fancy Indexing:", arr_stat[[0,2,4]]) # Output: [1 3 5]

# 8. Random Numbers
# rand(start, end, step) --> [0, 1] with step value
# randint(start, end, size) --> random integer with specified range --> [a, b]
print("Random floats:", np.random.rand(5))  # [0.37454012 0.95071431
0.73199394 0.59865848 0.15601864]
print("Random integers:", np.random.randint(1,10,5)) # [4 7 2 9 1]
print("Random normal:", np.random.randn(5))    # [-0.10321885 0.4105985
0.14404357 1.45427351 0.76103773]

# 9. Linear Algebra
# dot - dot product
# det - determinant
# inv - inverse
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print("Matrix Multiplication:", np.dot(A,B))   # [[19 22]
# [43 50]]
print("Determinant:", np.linalg.det(A))        # -2.0
print("Inverse:\n", np.linalg.inv(A))         # [[-2.  1. ]
# [ 1.5 -0.5]]

# 10. Broadcasting (allows arithmetic operations on arrays of different
shapes)
A = np.array([[1,2,3],[4,5,6]])
B = np.array([1,0,1])
print("Broadcasting A+B:", A+B)                # [[2 2 4]
# [5 5 7]]

# 11. File I/O (Save arrays to a file & load them later)
# 'arr.npy' - binary file that stores the NumPy array 'arr_save' exactly as it
is.
arr_save = np.array([1,2,3,4,5])
np.save('arr.npy', arr_save)                  # Saves array to file
'arr.npy'
loaded = np.load('arr.npy')
print("Loaded array:", loaded)                # [1 2 3 4 5]

```

Pandas

What is Pandas?

- Library used for working with datasets.
- It provides functions to analyze, clean, explore & manipulate data.

It mainly works with:

- **Series** → A **1D labelled array**. When placed inside a Data-Frame, it behaves like one column of data.
- **Data-Frame** → A **2D table of data**, made up of multiple Series (columns).

Installation → pip install pandas

Import → import pandas as pd

Series 1		Series 2		Series 3		Dataframe			
INDEX	DATA	INDEX	DATA	INDEX	DATA	INDEX	SERIES 1	SERIES 2	SERIES 3
0	A	0	1	0	[1, 2]	0	A	1	[1, 2]
1	B	1	2	1	A	1	B	2	A
2	C	2	3	2	1	2	C	3	1
3	D	3	4	3	(4, 5)	3	D	4	(4, 5)
4	E	4	5	4	{"a": 1}	4	E	5	{"a": 1}
5	F	5	6	5	6	5	F	6	6

Series

```
import pandas as pd
import numpy as np

# Default Syntax of Series
# pd.Series(
#     data=None,      # the actual values (list, array, dict, scalar, etc.)
#     index=None,     # row labels (default: 0,1,2,...)
#     dtype=None,     # force convert to any data type (int, float, string, etc.)
#     name=None,      # optional name for the Series
#     copy=False      # make a copy of data or not
# )

s = pd.Series(
    data=[10, 20, 30, np.nan, 20, 10, 40],      # values with NaN
    index=["A", "B", "C", "D", "E", "F", "G"],   # custom row labels
    dtype="float",                             # force convert to float type
    name="Marks",                              # name for Series
    copy=False                                 # no extra copy
)

print(s)    # print full Series

# A      10.0
# B      20.0
# C      30.0
# D       NaN
# E      20.0
# F      10.0
# G      40.0
# Name: Marks, dtype: float64
```

```

# -----
# 1. Basic Info
# -----
print("First 5 values (default):\n", s.head())
print("First 3 values:\n", s.head(3))
print("Last 5 values (default):\n", s.tail())
print("Last 2 values:\n", s.tail(2))
print("Row labels (index object):", s.index)
print("Row labels (index --> python list):", s.index.tolist())
print("Values only (NumPy array --> spaces):", s.values)
print("Values only (Python list --> commas):", s.values.tolist())

# First 5 values (default):
# A      10.0
# B      20.0
# C      30.0
# D       NaN
# E      20.0
# Name: Marks, dtype: float64

# First 3 values:
# A      10.0
# B      20.0
# C      30.0
# Name: Marks, dtype: float64

# Last 5 values (default):
# C      30.0
# D       NaN
# E      20.0
# F      10.0
# G      40.0
# Name: Marks, dtype: float64

# Last 2 values:
# F      10.0
# G      40.0
# Name: Marks, dtype: float64

# Row labels (index object): Index(['A', 'B', 'C', 'D', 'E', 'F', 'G'],
dtype='object')

# Row labels (index --> python list): ['A', 'B', 'C', 'D', 'E', 'F', 'G']

# Values only (NumPy array --> spaces): [10. 20. 30. nan 20. 10. 40.]

# Values only (Python list --> commas): [10.0, 20.0, 30.0, nan, 20.0, 10.0,
40.0]

```

```

# -----
# 2. Math / Statistics
# -----
print("Sum (ignores NaN):", s.sum())
print("Mean (average):", s.mean())
print("Maximum:", s.max())
print("Minimum:", s.min())
print("Standard Deviation:", s.std())

# Sum (ignores NaN): 130.0
# Mean (average): 21.666666666666668
# Maximum: 40.0
# Minimum: 10.0
# Standard Deviation: 11.547005383792516

# -----
# 3. Selection
# -----
print("First value:", s[0])
print("Slice values (s[1:3]):\n", s[1:3])
print("Select by label (s.loc['C']):", s.loc["C"])
print("Select by position (s.iloc[2]):", s.iloc[2])

# First value: 10.0
# Slice values (s[1:3]):
# B    20.0
# C    30.0
# Name: Marks, dtype: float64
# Select by label (s.loc['C']): 30.0
# Select by position (s.iloc[2]): 30.0

# -----
# 4. Cleaning
# -----
print("Remove missing (NaN) values:", s.dropna())
print("Fill missing (NaN) with 0:", s.fillna(0))
print("Replace 10 with 99:", s.replace(10, 99))

# Remove missing (NaN) values:
# A    10.0
# B    20.0
# C    30.0
# E    20.0
# F    10.0
# G    40.0
# Name: Marks, dtype: float64

```

```
# Fill missing (NaN) with 0:
# A    10.0
# B    20.0
# C    30.0
# D     0.0
# E    20.0
# F    10.0
# G    40.0
# Name: Marks, dtype: float64

# Replace 10 with 99:
# A    99.0
# B    20.0
# C    30.0
# D     NaN
# E    20.0
# F    99.0
# G    40.0
# Name: Marks, dtype: float64

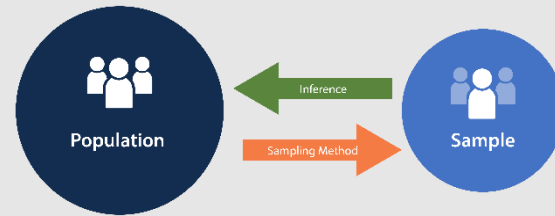
# -----
# 5. Other Useful
# -----
print("Frequency of each value:", s.value_counts())
print("Unique values:", s.unique())
print("Sorted values:", s.sort_values())

# Frequency of each value:
# 20.0    2
# 10.0    2
# 30.0    1
# 40.0    1
# Name: Marks, dtype: int64

# Unique values: [10. 20. 30. nan 40.]

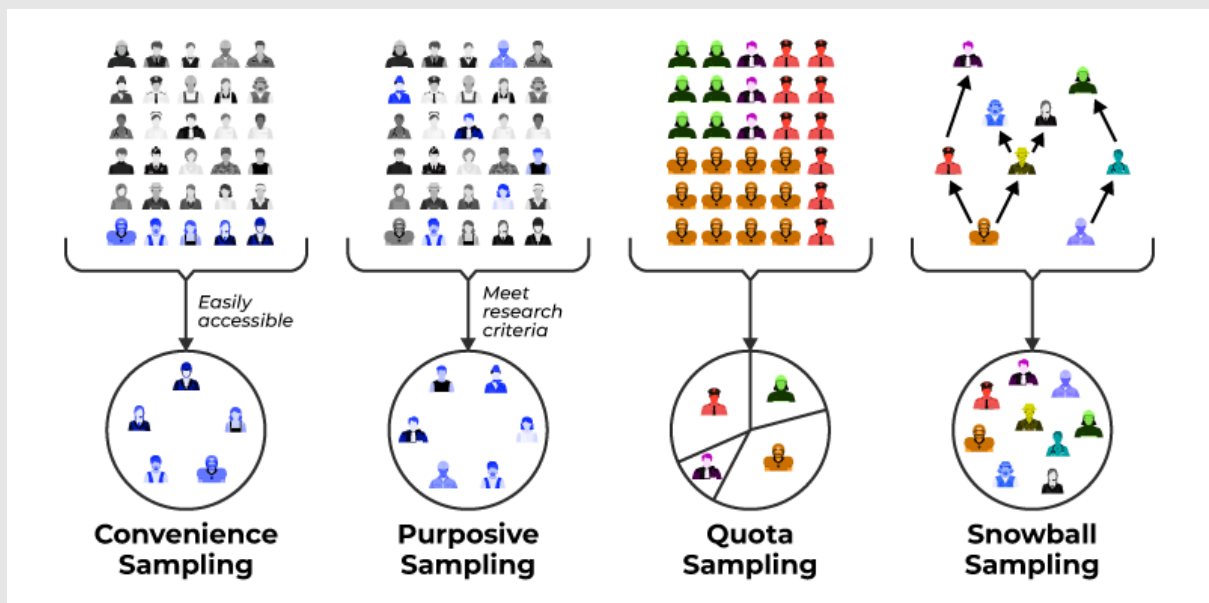
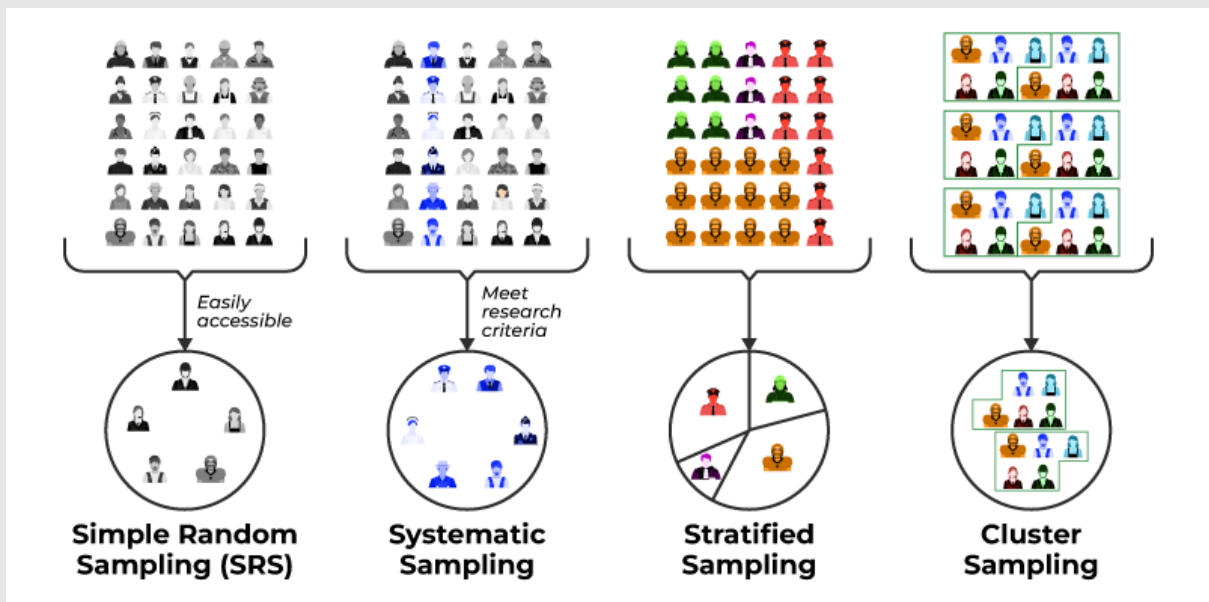
# Sorted values:
# A    10.0
# F    10.0
# B    20.0
# E    20.0
# C    30.0
# G    40.0
# D     NaN
# Name: Marks, dtype: float64
```

Statistical Inference



(1) Sampling

Term	Definition	Example
Population	The entire set of units in a study about which inferences (conclusions) are to be made.	All Amazon customers worldwide.
Sample	A finite subset of units selected from the population for observation or analysis, used to make inferences about the population.	1,000 Amazon customers chosen for a survey.
Sampling	The process (method) of selecting a sample from the population.	Randomly selecting 1,000 customers from the Amazon database.



Type	Definition	Example
Probability Sampling	A method where every unit in the population has a known & non-zero chance of being selected.	Amazon wants to survey customer satisfaction. They select customers using a method where every customer has a known, non-zero chance of being chosen.
Simple Random	Every unit in the population has an equal and independent chance of being selected.	Randomly picking 100 Amazon customers from the entire database.
Systematic	Every k-th unit from a sequentially ordered population is selected, starting from a random point.	Selecting every 50th Amazon order from the database.
Stratified	The population is divided into homogeneous subgroups (strata) , and samples (of one or more units) are randomly selected from each stratum.	Dividing Amazon customers by region (North, South, East, West) and randomly selecting from each region.
Cluster	Divide the population into groups (clusters) , pick a few groups randomly, and include all members of those groups in the sample.	Selecting 3 Amazon warehouses randomly and including all customers served by them.

Type	Definition	Example
Non-Probability Sampling	A method where every unit in the population does not have a known or guaranteed chance of being selected, and some units may have zero chance .	Amazon wants to survey customer satisfaction. They select customers who visit a particular store because it's easy to reach, so not every customer has a chance to be chosen.
Convenience	From the population, the researcher selects the sample that is easiest to reach (or contact) , not randomly.	From all women in the population, the researcher collects data only from those visiting a shopping mall (easy to access).
Purposive (Judgemental)	From the population, the researcher purposely selects the most relevant units for the sample.	From all FMCG companies, the sample includes only Nestle and Hindustan Unilever because they are important.
Quota	The population is divided into groups (age, gender etc). A fixed quota/sample (of one or more units) is taken from each group.	From the total population, divide by gender and select 15 men + 15 women for the sample.
Snowball	From the population, a few initial people are chosen. Then they help recruit more from the same population. The sample grows step by step.	From the homeless population, 1 person is chosen. He refers 2 more, they refer others, and the sample keeps expanding.

When to Use Probability Sampling

- **Simple Random:** When every member has an equal chance (e.g., lottery method).
- **Stratified:** When the population has subgroups and you need proportional representation.
- **Systematic:** When population list is available and you can pick every k^{th} element.
- **Cluster:** When population is widespread and natural groups (clusters) exist, useful for cost-saving.

When to Use Non-Probability Sampling

- **Convenience:** Quick, inexpensive preliminary research
- **Purposive:** When you need experts or specific characteristics
- **Snowball:** When population is hard to reach (e.g., niche groups)
- **Quota:** When you need to ensure certain subgroups are represented

Advantages & Disadvantages of Sampling Methods

Probability Sampling

1. Simple Random Sampling

- ☒ *Advantage:* Unbiased, each unit has equal chance.
- ☒ *Disadvantage:* Needs complete population list, can be costly for large populations.

2. Systematic Sampling

- ☒ *Advantage:* Simple, easy to implement, ensures spread across population.
- ☒ *Disadvantage:* Risk of bias if population has hidden patterns (periodicity).

3. Stratified Sampling

- ☒ *Advantage:* Ensures representation of all subgroups (strata), increases accuracy.
- ☒ *Disadvantage:* Needs knowledge of strata beforehand, more complex.

4. Cluster Sampling

- ☒ *Advantage:* Cost-effective, practical for large populations spread across areas.
- ☒ *Disadvantage:* Less accurate, may increase sampling error if clusters not representative.

Non-Probability Sampling

1. Convenience Sampling

-  *Advantage*: Quick, cheap, easy.
-  *Disadvantage*: Highly biased, not generalizable.

2. Purposive Sampling

-  *Advantage*: Focused on specific, relevant units.
-  *Disadvantage*: Researcher bias, may ignore other important units.

3. Snowball Sampling

-  *Advantage*: Useful for hidden or hard-to-reach populations.
-  *Disadvantage*: Bias due to social connections, not random.

4. Quota Sampling

-  *Advantage*: Ensures representation of certain groups.
-  *Disadvantage*: Not random, researcher may select biased samples within quotas.

When to Use Probability vs Non-Probability Sampling

Probability Sampling (Best for Accuracy & Generalization)

- When population is **large and well-defined**.
- When a **complete list (sampling frame)** is available.
- When you want **unbiased and statistically valid results**.
- Example: National survey, company-wide customer satisfaction survey.

Non-Probability Sampling (Best for Practicality & Speed)

- When population is **hard to define** (hidden groups).
- When you want **quick, low-cost preliminary research**.
- When statistical generalization is **not the main goal**.
- Example: Interviewing mall visitors, expert panels, niche community research.

1. Systematic Sampling - Key Detail

Risk of bias if the population has hidden patterns (periodicity).
(You mentioned systematic sampling but didn't note this disadvantage.)

2. Cluster Sampling - Key Detail

Less accurate if selected clusters are not representative.
(Your notes mention cost-effectiveness, but not potential inaccuracy.)

3. Convenience Sampling - Key Detail

Highly biased and not generalizable.
(You wrote when to use, but the disadvantage is missing.)

4. Purposive Sampling - Key Detail

Risk of researcher bias and ignoring other important units.

5. Snowball Sampling - Key Detail

Bias due to social connections; not random.

6. Quota Sampling - Key Detail

Not random; researcher may select biased samples within quotas.

7. Short Python Tips / Reminders

Systematic sampling can be done with: `df.iloc[::k]`

Cluster sampling: randomly select clusters, then include all their members
(Your notes have examples, but this concise "how-to" tip is missing.)

Python Code Reminders for Missing Points

1. **Systematic Sampling** - pick every k^{th} element:

```
k = 5
```

```
systematic_sample = df.iloc[::k]
```

2. **Cluster Sampling** - select clusters randomly, then all their members:

```
clusters = df['cluster_column'].unique()
```

```
selected_clusters = np.random.choice(clusters, size=2, replace=False)
```

```
cluster_sample = df[df['cluster_column'].isin(selected_clusters)]
```

3. **Quota Sampling (concise way)** - sample proportionally from groups:

```
quota_sample = pd.DataFrame()
```

```
for group, prop in group_proportions.items():
```

```
    group_data = df[df['group_col'] == group]
```

```
    n_samples = int(total_sample_size * prop)
```

```
    sampled = group_data.sample(n_samples, random_state=42)
```

```
    quota_sample = pd.concat([quota_sample, sampled])
```

```

import pandas as pd
import numpy as np

# Create dataset
np.random.seed(42)
prob_data = {
    'id': range(1, 101),
    'age': np.random.randint(18, 65, 100),
    'income': np.random.normal(50000, 15000, 100),
    'region': np.random.choice(['North', 'South', 'East', 'West'], 100)
}
df1 = pd.DataFrame(prob_data)

print("FULL POPULATION (100 records):")
print(df1.head())
print(f"Total records: {len(df1)}\n")

#    id  age      income region
# 0   1   54  67399.342077  South
# 1   2   46  45236.428568  North
# 2   3   58  48492.029384   East
# 3   4   39  52813.975883   West
# 4   5   29  50888.162091  South
# Total records: 100

# 1. Simple Random Sampling
simple_random = df1.sample(frac=0.1, random_state=42)
print("SIMPLE RANDOM SAMPLE (10%):")
print(simple_random.head())
print(f"Sample size: {len(simple_random)}\n")
(first 5 rows)
#    id  age      income region
# 83  84   22  51042.335171   East
# 53  54   43  48588.672829  South
# 70  71   19  52923.289231  West
# 25  26   23  47236.456823  North
# 44  45   54  48391.728492  South
# Sample size: 10

```

```

# 2. Stratified Sampling
stratified_sample = df1.groupby('region', group_keys=False).apply(
    lambda x: x.sample(frac=0.1, random_state=42)
)
print("STRATIFIED SAMPLE (10% from each region):")
print(stratified_sample.head())
print(f"Sample size: {len(stratified_sample)}\n")

(first 5 rows)
#      id  age      income region
# 5      6   33  52332.828192  North
# 29     30   44  50122.384921  North
# 8       9   60  48122.912398   East
# 39     40   28  50123.221342   East
# 53     54   43  48588.672829  South
# Sample size: 10

# 3. Systematic Sampling
k = 5
systematic_sample = df1.iloc[::k]
print("SYSTEMATIC SAMPLE (every 5th record):")
print(systematic_sample.head())
print(f"Sample size: {len(systematic_sample)}\n")

#      id  age      income region
# 0      1   54  67399.342077  South
# 5      6   33  52332.828192  North
# 10     11   60  47429.111223   East
# 15     16   24  49122.221332   West
# 20     21   46  48722.442112  North
# Sample size: 20

# 4. Cluster Sampling
clusters = df1['region'].unique()
selected_clusters = np.random.choice(clusters, size=2, replace=False)
cluster_sample = df1[df1['region'].isin(selected_clusters)]
print(f"CLUSTER SAMPLE (clusters: {selected_clusters}):")
print(cluster_sample.head())
print(f"Sample size: {len(cluster_sample)}\n")
(first 5 rows, example if clusters=['West', 'South'])
#      id  age      income region
# 0      1   54  67399.342077  South
# 3      4   39  52813.975883   West
# 4      5   29  50888.162091  South
# 15     16   24  49122.221332   West
# 17     18   50  47892.339123   West
# Sample size: 48

```

```

# Create dataset
np.random.seed(42)
nonprob_data = {
    'customer_id': range(1, 101),
    'age': np.random.randint(18, 70, 100),
    'satisfaction': np.random.randint(1, 11, 100),
    'loyalty_years': np.random.randint(0, 10, 100),
    'region': np.random.choice(['North', 'South', 'East', 'West'], 100)
}
df2 = pd.DataFrame(nonprob_data)

print("FULL POPULATION (100 customers):")
print(df2.head())
print(f"Total records: {len(df2)}\n")

#      customer_id  age  satisfaction  loyalty_years region
# 0              1   58              7              7 North
# 1              2   24              4              3 West
# 2              3   62              8              6 East
# 3              4   43              5              0 South
# 4              5   29              9              2 North
# Total records: 100

# 1. Convenience Sampling
convenience_sample = df2.head(20)
print("CONVENIENCE SAMPLE (first 20 customers):")
print(convenience_sample.head())
print(f"Sample size: {len(convenience_sample)}\n")
#      customer_id  age  satisfaction  loyalty_years region
# 0              1   58              7              7 North
# 1              2   24              4              3 West
# 2              3   62              8              6 East
# 3              4   43              5              0 South
# 4              5   29              9              2 North
# Sample size: 20

# 2. Purposive Sampling
purposive_sample = df2[df2['satisfaction'] >= 8]
print("PURPOSIVE SAMPLE (satisfaction >= 8):")
print(purposive_sample.head())
print(f"Sample size: {len(purposive_sample)}\n")
#      customer_id  age  satisfaction  loyalty_years region
# 2              3   62              8              6 East
# 4              5   29              9              2 North
# 8              9   20             10              0 West
# 11             12   24              8              7 South
# 15             16   42              9              1 West
# Sample size: 38

```

```

# 3. Snowball Sampling
seed_ids = [1, 10, 20, 30, 40]
snowball_sample = df2[df2['customer_id'].isin(seed_ids)]
for seed in seed_ids:
    referrals =
df2[~df2['customer_id'].isin(snowball_sample['customer_id'])].sample(2)
    snowball_sample = pd.concat([snowball_sample, referrals])

print("SNOWBALL SAMPLE (seed + referrals):")
print(snowball_sample.head())
print(f"Sample size: {len(snowball_sample)}\n")
#   customer_id  age  satisfaction  loyalty_years  region
# 0             1   58             7              7  North
# 9             10   23             4              2  East
# 19            20   34             6              1  South
# 29            30   46             3              8  North
# 39            40   55             5              6  East
# Sample size: 15

# 4. Quota Sampling
regional_proportions = df2['region'].value_counts(normalize=True)
quota_sample = pd.DataFrame()
for region, prop in regional_proportions.items():
    region_data = df2[df2['region'] == region]
    n_samples = int(50 * prop) # total sample = 50
    sampled = region_data.sample(n_samples, random_state=42)
    quota_sample = pd.concat([quota_sample, sampled])

print("QUOTA SAMPLE (regional representation, total 50):")
print(quota_sample.head())
print(f"Sample size: {len(quota_sample)}")
print("Regional distribution in sample:")
print(quota_sample['region'].value_counts())
#   customer_id  age  satisfaction  loyalty_years  region
# 1             2   24             4              3  West
# 0             1   58             7              7  North
# 3             4   43             5              0  South
# 2             3   62             8              6  East
# 6             7   39             3              2  West
# Sample size: 50
# Regional distribution in sample:
# West      14
# North     13
# East      13
# South     10

```

(2) Hypothesis Testing

Statistical Method - way to collect, organize, analyze, and interpret data to make decisions (draw conclusions) about a population.

Terms	Description	Example
Hypothesis Testing	A statistical method used to decide whether a claim or assumption about a population is likely to be true or false, based on sample data.	Testing if the average marks of the class = 70 using the sample of 20 students.
Hypothesis	A statement (assumption) that we want to test about the population.	Average marks of the class = 70.
Null Hypothesis (H_0)	Default assumption (the teacher's claim is true).	H_0 : Average marks = 70
Alternative Hypothesis (H_a/H_1)	The statement opposite to H_0 that we want to prove.	H_a : Average marks $\neq$ 70
Collect Data	Take a sample marks of 20 students from the population.	Sample marks = [68, 70, 72, 65, ...] Sample average = 72
P-value $\in [0, 1]$	The probability of obtaining the observed sample result (or something more extreme), if the null hypothesis (H_0) is true.	P-value $\approx 0.04 \rightarrow$ There is a 4% chance of getting a sample average of ≥ 72 or ≤ 68 , if the true class average is 70.
Significance Level $\alpha \in [0, 1]$	- Probability of rejecting the null hypothesis (H_0) when H_0 is actually true. - It is a threshold (cutoff) used to decide whether the sample data is "unusual enough" (extreme/rare) to reject H_0.	Decision Rule: If P-value $\leq \alpha \rightarrow$ Reject H_0 If P-value $> \alpha \rightarrow$ Accept H_0 P-value = 0.04 < 0.05, we reject H_0 .
Conclusion	The final decision after comparing P-value with α .	The sample provides enough evidence to conclude that the average marks of the class are different from 70.

Types of Hypothesis Tests (based on Alternative Hypothesis H_a)

- μ = population mean.
- value = the number claimed in H_0 (70).

Type of Test	Alternative Hypothesis (H_a)	Key Question	Rejection Region	Example
One-Tailed Test (General)	$H_a: \mu > \text{value}$ or $\mu < \text{value}$	"Do we care about change in only one direction (increase <i>or</i> decrease)?"	One side of the distribution (right or left)	New medicine lowers blood pressure. $H_0: \mu = 120$ mg $H_a: \mu < 120$ mg
Right-Tailed Test	$H_a: \mu > \text{value}$	"Is the mean significantly greater?"	Right tail (large values)	Fertilizer increases plant height. $H_0: \mu = 10$ cm $H_a: \mu > 10$ cm
Left-Tailed Test	$H_a: \mu < \text{value}$	"Is the mean significantly smaller?"	Left tail (small values)	New diet reduces weight. $H_0: \mu = 0$ kg $H_a: \mu < 0$ kg
Two-Tailed Test	$H_a: \mu \neq \text{value}$	"Is the mean significantly different (either higher or lower)?"	Both tails (large $\uparrow$ or small $\downarrow$)	Machine out of calibration. $H_0: \mu = 16$ oz $H_a: \mu \neq 16$ oz

Rule of Thumb

- If we care about change in **only one direction (increase *or* decrease)**, use **one-tailed**.
- If we only care about **increase**, use right-tailed.
- If we only care about **decrease**, use left-tailed.
- If we care about **any difference**, use two-tailed.

Type I & Type II Errors

When making decisions using hypothesis testing, **mistakes can happen**.

Decision	Reality: H_0 True	Reality: H_0 False
Accept H_0	✓ Correct decision	✗ Type II Error (β) = fail to detect the truth
Reject H_0	✗ Type I Error (α) = false alarm	✓ Correct decision

Type I Error (False Positive)

- **Definition:** Rejecting H_0 when it is actually true.
- **Symbol:** α (same as significance level).
- **Example:** Teacher claims class average = 70.
- You reject H_0 and say “average $\neq 70$ ” even though the real class average is **actually 70**.

Type II Error (False Negative)

- **Definition:** Accepting H_0 when it is actually false.
- **Symbol:** β .
- **Example:** Teacher claims average = 70.
- You accept H_0 (say average = 70) even though the true class average is **actually 75**.

Connection Between α , β , and Power

- **α (Type I error):** Probability of wrongly rejecting H_0 .
- **β (Type II error):** Probability of wrongly accepting H_0 .
- **Power of a Test = $1 - \beta$:** Probability of correctly rejecting H_0 when it is false.

For a Good Test

- **α low** (avoid false alarms)
- **power high** (avoid missing real effects).

Formal definition / purpose of hypothesis testing – “Hypothesis testing is a formal way to make a decision between two competing ideas using data. It answers the question: *Is the evidence from my sample strong enough to conclude something about the whole population?*”

Step-by-step example with the magic coin – The detailed example illustrating:

- How to state H_0 and H_a for a biased coin
- How to calculate P-value with a sample of 100 flips
- How to make decisions based on the P-value compared to α
- What happens if P-value is higher (e.g., 0.16)

(3) Outlier

A data point that is very different (much higher or much lower) compared to the rest of the data.

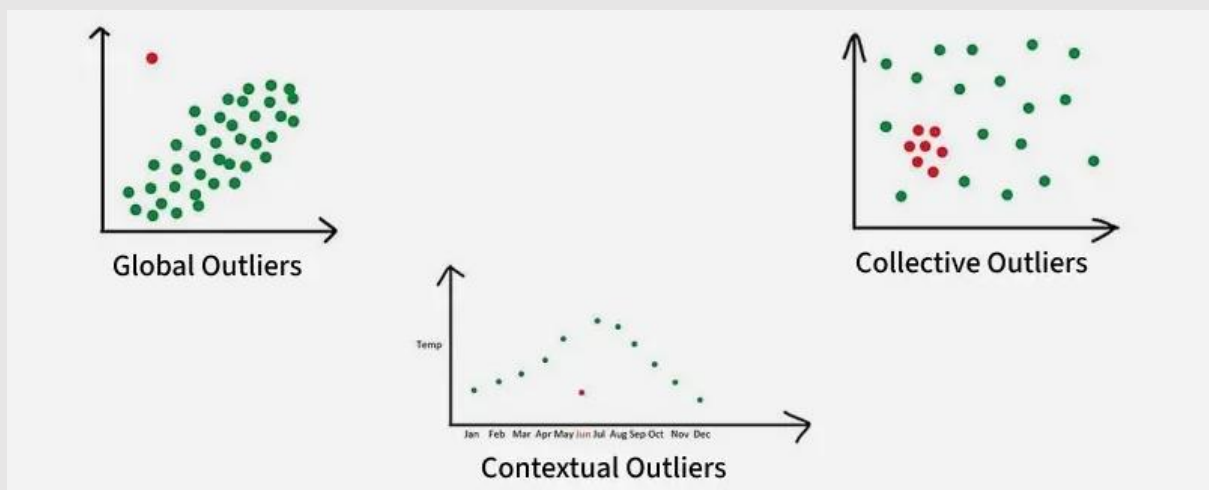
Example: Marks of 10 students: [70, 72, 68, 74, 71, 69, 73, 72, 10, 100]

- Most marks are around **70–74**.
- But **10** (very low) & **100** (very high) are **outliers**.

Why Outliers Happen?

- **Measurement Error** → e.g., a faulty sensor shows temperature = 500°C instead of 50°C.
- **Data Entry Error** → e.g., typing "1000" instead of "100" for marks.
- **Genuine Extreme Values** → e.g., in a class test, most students score 60–80, but one genius scores 100 & one very weak student scores 10.

Type of Outlier	Definition	Example
Global Outlier (Point Anomaly)	A single data point that is very different from all other points in the dataset.	Marks: [70, 72, 71, 69, 150] → 150
Contextual Outlier (Conditional Anomaly)	A data point that is unusual only in a certain context .	Temperature: [38, 39, 41, 40] → 40°C is high in winter (may be normal in summer)
Collective Outlier (Group Anomaly)	A group of data points that together are unusual, even if individual points don't look abnormal.	Stock prices: [100, 101, 102, 103, 200, 201, 202] → [200, 201, 202]



Outlier Detection

It is the process of finding data points that are very different from most of the other data.

Why Detect Outliers?

- Outliers can **affect (increase/decrease) & distort (mislead)** statistical results. For example, they can change the mean, median, or S.D., which can make our analysis less accurate. By finding & handling outliers, we make sure our data analysis is correct & reliable.
 - Outliers can significantly affect mean and standard deviation; median is more robust to outliers.
1. **Improve Accuracy:** Removing or handling outliers helps data models give better predictions.
 2. **Fraud Detection:** Outliers can show unusual or suspicious activity, like in financial transactions.
 3. **Maintain Data Quality:** Detecting outliers keeps your data clean and trustworthy.
 4. **Better Model Performance:** Outliers can affect statistical or machine learning models, so handling them improves model accuracy.
 5. **Discover Insights:** Sometimes outliers show interesting trends, rare events, or opportunities that normal data doesn't reveal.

Methods to Detect Outliers

(1) Standard Deviation

- It is a statistical approach for detecting outliers.
- It assumes the data follows a **normal distribution**.
- Any data point lying far from the mean is flagged as an outlier.
- **Key idea:** Points outside **±3 standard deviations** from the mean are considered outliers (sometimes ±2 is used for stricter detection).
- **Application:** Works best for **univariate data analysis**, where only one variable (e.g., height, weight, marks) is analyzed at a time.

Dataset: {1, 2, 2, 3, 1, 3, 10}

x_i	$x_i - \mu$	$(x_i - \mu)^2$
1	-2.14	4.57
2	-1.14	1.30
2	-1.14	1.30
3	-0.14	0.02
1	-2.14	4.57
3	-0.14	0.02
10	6.86	47.02
$\Sigma x_i = 22$		$\Sigma (x_i - \mu)^2 = 58.80$

- **Mean (μ)** = $\frac{\Sigma x_i}{n} = \frac{22}{7} = 3.14$
- **Standard Deviation (σ)** = $\sqrt{\frac{\Sigma (x_i - \mu)^2}{n}} = \sqrt{\frac{58.80}{7}} = 2.90$
- **Bounds:** $\mu \pm 2\sigma$ (or) $\mu \pm 3\sigma$

✓ **2 σ rule → Points outside [-2.66, 8.94] are outliers → Outlier = 10**

$$\text{Lower} = \mu - 2\sigma = 3.14 - 2(2.90) = -2.66$$

$$\text{Upper} = \mu + 2\sigma = 3.14 + 2(2.90) = 8.94$$

✓ **3 σ rule → Points outside [-5.56, 11.84] are outliers → No Outlier**

$$\text{Lower} = \mu - 3\sigma = 3.14 - 3(2.90) = -5.56$$

$$\text{Upper} = \mu + 3\sigma = 3.14 + 3(2.90) = 11.84$$

(2) IQR (Interquartile Range)

- The IQR method identifies outliers based on the **middle 50% of the data**.
- This method is **robust to extreme values**.
- Suitable for **skewed or non-normal datasets**, where the spread of the middle 50% is more meaningful than mean & standard deviation.

Calculation

- **$IQR = Q3 - Q1$**

where **$Q1$** = 25th percentile = Median of First Half

$Q3$ = 75th percentile = Median of Second Half

- Lower Bound = $Q1 - 1.5(IQR)$
- Upper Bound = $Q3 + 1.5(IQR)$

Example: Dataset $X = \{3, 5, 7, 9, 11, 13, 30\}$, find outlier using the IQR method.

- *$Q1$ (25th percentile): Median of first half = Median of $\{3, 5, 7\} = 5$*
 - *$Q3$ (75th percentile): Median of second half = Median of $\{11, 13, 30\} = 13$*
- $IQR = Q3 - Q1 = 13 - 5 = 8$*
- Lower Bound: $Q1 - 1.5 \times IQR = 5 - 1.5 \times 8 = 5 - 12 = -7$*
- Upper Bound: $Q3 + 1.5 \times IQR = 13 + 1.5 \times 8 = 13 + 12 = 25$*
- Therefore the interval is -7 to 25. 30 lies outside the interval, therefore is an outlier.*

(3) Z-Score

- It is a statistical technique to detect outliers based on how far data points lie from the mean.
- It measures this distance in terms of **SD**.
- A data point's **Z-score** tells how many SDs it is above (+) or below (-) the mean.
- Assumes data follows an **approximately normal distribution**.
- Works best with **large datasets**; can be sensitive to extreme values in small datasets.

Calculation

- **Mean:** μ
- **Standard Deviation:** σ
- **Z-score for Each Value:** $Z = \frac{x - \mu}{\sigma}$
- **Threshold Rule**

- **Mild Outlier:** $|Z| > 2 \rightarrow |Z| = (2, 3]$
- **Extreme Outlier:** $|Z| > 3 \rightarrow |Z| > 3$

Dataset: $x = \{4, 5, 5, 6, 7, 8, 20\}$

(1) Mean $\rightarrow \mu \approx 7.857$

(2) Standard Deviation $\rightarrow \sigma \approx 5.111$

(3) Z-Scores

- $Z(4) \approx -0.755$
- $Z(5) \approx -0.559$
- $Z(5) \approx -0.559$
- $Z(6) \approx -0.363$
- $Z(7) \approx -0.168$
- $Z(8) \approx 0.028$
- $Z(20) \approx 2.376$

(4) Apply Threshold

- $Z(4) \approx -0.755 < 2 \rightarrow$ not an outlier
- $Z(5) \approx -0.559 < 2 \rightarrow$ not an outlier
- $Z(5) \approx -0.559 < 2 \rightarrow$ not an outlier
- $Z(6) \approx -0.363 < 2 \rightarrow$ not an outlier
- $Z(7) \approx -0.168 < 2 \rightarrow$ not an outlier
- $Z(8) \approx 0.028 < 2 \rightarrow$ not an outlier
- $Z(20) \approx 2.376 > 2 \rightarrow$ mild outlier (but not extreme since $|Z| < 3$)

Outlier Detection Using Box Plot, Scatter Plot, Z-Score & IQR Methods Visually

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# ---- Sample Data ----
data_univariate = np.array([25, 28, 29, 30, 31, 32, 33, 34, 100]) # For Box,
Z-Score, IQR
x_bivariate = np.array([1,2,3,4,5,6,7,8,9,10])
y_bivariate = np.array([2,4,5,4,6,8,7,10,9,100]) # For Scatter Plot

# ---- 1. Box Plot ----
plt.figure(figsize=(6,4))
sns.boxplot(x=data_univariate)
plt.title('Box Plot for Outlier Detection')
plt.show()

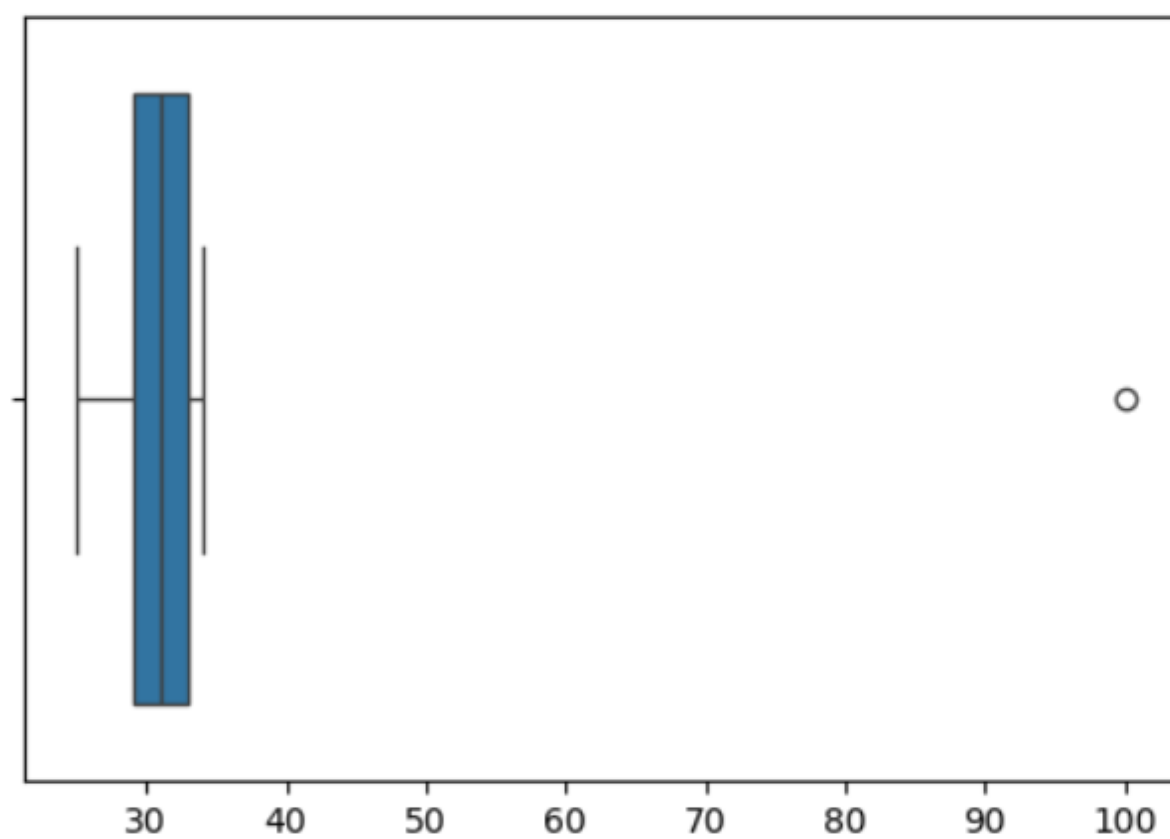
# ---- 2. Scatter Plot ----
plt.figure(figsize=(6,4))
plt.scatter(x_bivariate, y_bivariate)
plt.title('Scatter Plot for Outlier Detection')
plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True)
plt.show()

# ---- 3. Z-Score Method ----
z_scores = np.abs(stats.zscore(data_univariate))
threshold = 3
outliers_z = data_univariate[z_scores > threshold]
print("Outliers detected with Z-Score Method:", outliers_z)

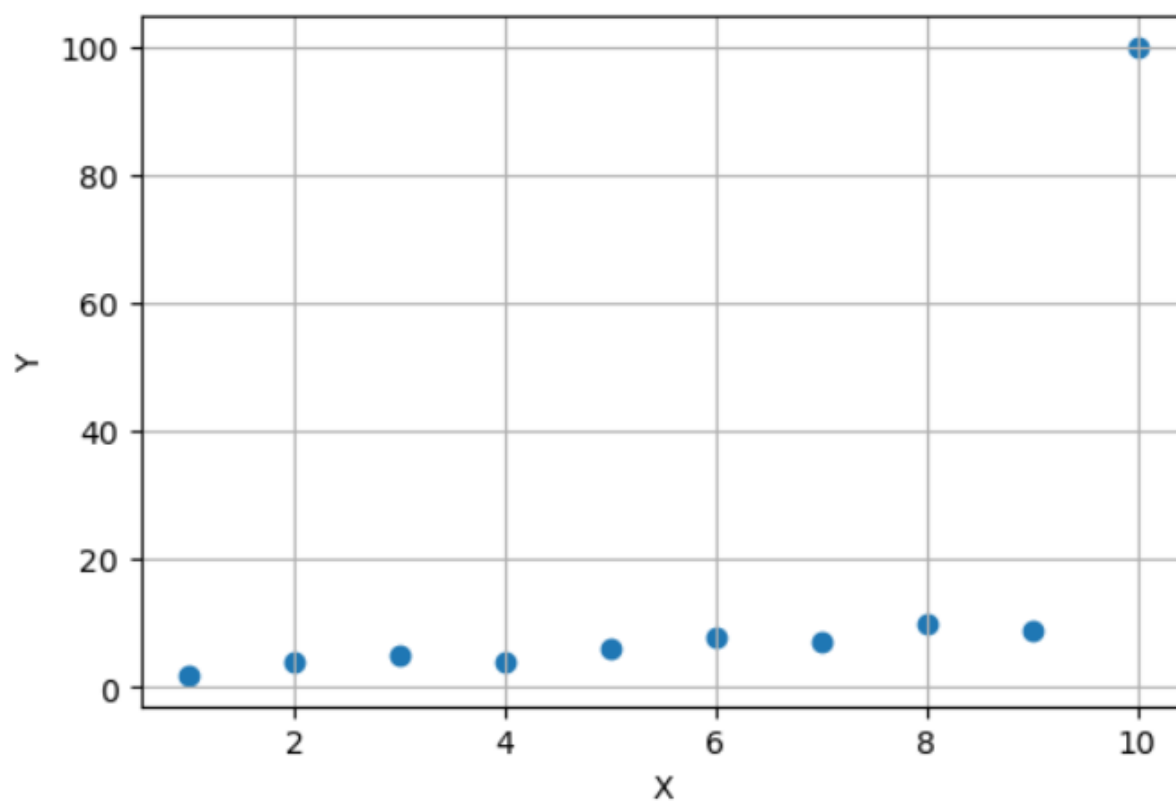
# ---- 4. IQR Method ----
Q1 = np.percentile(data_univariate, 25)
Q3 = np.percentile(data_univariate, 75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers_iqr = data_univariate[(data_univariate < lower_bound) |
(data_univariate > upper_bound)]
print("Outliers detected with IQR Method:", outliers_iqr)

#Outliers detected with Z-Score Method: []
#Outliers detected with IQR Method: [100]
```

Box Plot for Outlier Detection



Scatter Plot for Outlier Detection



Handling Methods

	Trimming / Removing Outliers	Quantile-Based Flooring & Capping (Winsorization)	Mean/Median Imputation
Definition	Simply remove the detected outliers.	Replace outliers with percentile thresholds (e.g., 10th & 90th).	Replace outliers with median (preferred) .
When to use	When outliers are errors & dataset is large enough .	When you want to keep all data points but reduce extreme influence.	When dataset must remain complete (e.g., ML models).

Handling Method	Explanation	When to Use
Trimming / Removing Outliers	This is the simplest method. You completely remove the outlier data points from the dataset. This is a good approach when you are sure the outliers are due to errors and you have a large enough dataset that losing a few points won't affect the analysis.	When outliers are known errors and the dataset size remains statistically significant after removal.
Quantile-Based Flooring & Capping (Winsorization)	Instead of removing data, you bring the outliers closer to the rest of the data. You set a "floor" (minimum value) and a "cap" (maximum value). Any point below the floor is raised to the floor value; any point above the cap is reduced to the cap value. Common choices are the 5th & 95th percentiles or 10th & 90th percentiles.	When you want to retain all data points but minimize the extreme influence of outliers on statistical models.
Mean / Median Imputation	You replace the outlier values with a central tendency measure—usually the median. The mean is not recommended because it itself is skewed by outliers. The median is robust to outliers.	When you need to preserve the dataset size for algorithms that require complete data and the outliers are numerical errors.

```

import numpy as np

# ---- Sample Dataset ----
sample = np.array([15, 20, 18, 7, 13, 16, 11, 101, 2, 15, 10, 9])
sample_outliers = [2, 101] # assuming detected outliers

# ---- 1. Trimming / Removing Outliers ----
a = sample.copy()
for i in sample_outliers:
    a = np.delete(a, np.where(a == i))
print("\nAfter Trimming:", a)

# ---- 2. Quantile-Based Flooring & Capping (Winsorization) ----
tenth_percentile = np.percentile(sample, 10)
ninetieth_percentile = np.percentile(sample, 90)
b = np.clip(sample, tenth_percentile, ninetieth_percentile)
print("After Capping/Flooring:", b)

# ---- 3. Mean/Median Imputation ----
median_value = np.median(sample)
c = sample.copy()
for i in sample_outliers:
    c[c == i] = median_value
print("After Median Imputation:", c)

#Original Data: [ 15  20  18   7  13  16  11 101   2  15  10   9]
#Detected Outliers: [2, 101]

#After Trimming: [15 20 18  7 13 16 11 15 10  9]
#After Capping/Flooring: [15. 20. 18.   7. 13. 16. 11. 47.   7. 15. 10.   9.]
#After Median Imputation: [15 20 18  7 13 16 11 15 15 10  9]

```

(4) Understanding & Plotting Distributions (Univariate, Bivariate, Multivariate Statistical Analysis)

Understanding Distributions

- A distribution describes **how the values of a variable are spread out or clustered together**.
- It tells you:
 - What values are **common**
 - What values are **rare**
 - The **overall shape** of the data

Importance of understanding distributions:

- **Summarizes Data:** Provides a more powerful summary than just the mean or median.
- **Informs Modeling:** Many statistical techniques (like linear regression) assume data follows a specific distribution (e.g., normal distribution).
- **Enables Prediction:** Understanding the distribution allows calculation of probabilities and inference about future observations.

Types of Data & Analysis

1. Univariate Data

- **Definition:** Data with only **one variable** per observation. Focuses on measuring a single characteristic.
- **Example:** Heights (in cm) of seven students:
- 164, 167.3, 170, 174.2, 178, 180, 186

Key Points:

1. **No Relationships:** Focuses on one variable; does not consider interactions.
2. **Descriptive Statistics:**
 - **Measures of Central Tendency:** Mean, Median, Mode
 - **Measures of Dispersion:** Range, Variance, Standard Deviation
3. **Visualization:** Histograms, box plots, and other graphical methods.

Python Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Dataset
heights = [164, 167.3, 170, 174.2, 178, 180, 186]
df = pd.DataFrame(heights, columns=['Height (cm)'])
print("Dataset:")
```

```

print(df)

# Descriptive statistics
print("\n--- DESCRIPTIVE STATISTICS ---")
print(f"Mean: {df['Height (cm)'].mean():.2f} cm")
print(f"Median: {df['Height (cm)'].median()} cm")
print(f"Mode: {df['Height (cm)'].mode().values} cm")
print(f"Range: {df['Height (cm)'].max() - df['Height (cm)'].min():.1f} cm")
print(f"Variance: {df['Height (cm)'].var():.2f} cm2")
print(f"Standard Deviation: {df['Height (cm)'].std():.2f} cm")
print(f"25th Percentile (Q1): {df['Height (cm)'].quantile(0.25)} cm")
print(f"75th Percentile (Q3): {df['Height (cm)'].quantile(0.75)} cm")
print(f"IQR (Q3 - Q1): {df['Height (cm)'].quantile(0.75) - df['Height (cm)'].quantile(0.25):.2f} cm")

# Complete summary
print("\n" + "="*50)
print("Complete Summary with .describe():")
print(df['Height (cm)'].describe())

# Plots
sns.set_style("whitegrid")
plt.figure(figsize=(15, 10))

# Histogram with KDE
plt.subplot(2, 2, 1)
sns.histplot(data=df, x='Height (cm)', kde=True, bins=5, color='skyblue')
plt.axvline(df['Height (cm)'].mean(), color='red', linestyle='--', label=f'Mean: {df["Height (cm)"].mean():.1f}cm')
plt.axvline(df['Height (cm)'].median(), color='green', linestyle='--', label=f'Median: {df["Height (cm)"].median()}cm')
plt.title('Distribution of Student Heights (Histogram)')
plt.legend()

# Box Plot
plt.subplot(2, 2, 2)
sns.boxplot(data=df, y='Height (cm)', color='lightcoral')
plt.title('Distribution of Student Heights (Box Plot)')

plt.tight_layout()
plt.show()

```

2. Bivariate Data

- **Definition:** Each observation has **two variables**. Used to understand relationships between variables.
- **Example:** Temperature and Ice Cream Sales
- Temperature: 20, 25, 30, 35
- Ice Cream Sales: 2000, 2500, 3500, 5000

Key Points:

1. **Relationship Analysis:** Positive, negative, or no clear pattern.
2. **Correlation:** Measures strength and direction of relationship (-1 to +1).
3. **Visualization:** Scatter plots to visualize relationships.

Python Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

# Data
data = {'Temperature': [20, 25, 30, 35, 40],
        'Sales': [2000, 2500, 3500, 5000, 6000]}
df = pd.DataFrame(data)

# Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(df['Temperature'], df['Sales'], color='blue', s=100)
plt.title('Ice Cream Sales vs Temperature')
plt.xlabel('Temperature (°C)')
plt.ylabel('Sales ($)')
plt.grid(True)
plt.show()

# Correlation
correlation = df['Temperature'].corr(df['Sales'])
print(f"Correlation coefficient: {correlation:.3f}")
```

3. Multivariate Data

- **Definition:** Observations have **more than two variables**. Focuses on interactions among multiple variables.
- **Example:** Advertisement clicks based on ad type and gender

Advertisement	Gender	Click rate
Ad1	Male	80
Ad3	Female	55
Ad2	Female	123
Ad1	Male	66
Ad3	Male	35

Key Points:

1. **Complex Relationships:** Finds patterns not obvious in individual variables.
2. **Analysis Techniques:** Regression analysis, PCA, MANOVA.
3. **Interpretation:** Understands interactions among variables.

Python Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

# Data
data = {'Ad': ['Ad1', 'Ad3', 'Ad2', 'Ad1', 'Ad3'],
        'Gender': ['Male', 'Female', 'Female', 'Male', 'Male'],
        'Clicks': [80, 55, 123, 66, 35]}
df = pd.DataFrame(data)
print("DATA:")
print(df)

# Grouped bar chart
avg_clicks = df.groupby(['Ad', 'Gender'])['Clicks'].mean().unstack()
avg_clicks.plot(kind='bar', color=['lightblue', 'pink'])
plt.title('Average Clicks by Ad and Gender')
plt.xlabel('Advertisement')
plt.ylabel('Average Click Rate')
plt.legend(title='Gender')
plt.xticks(rotation=0)
plt.show()
```

4. Real-World Applications

Type of Analysis	Applications
Univariate	Customer behavior (age, spending patterns), financial reporting
Bivariate	Market research (ads vs sales), customer satisfaction (ratings vs purchase frequency)
Multivariate	Healthcare predictive modeling (age, medical history, lifestyle), financial risk analysis (interest rates, inflation, market conditions)

Data Visualization

Data Visualization is the graphical representation of data to help identify patterns, trends & insights. It makes data easy to understand & communicate.

Common Python libraries

Matplotlib	Basic plotting library for 2D plots.
Seaborn	Built on Matplotlib, easier for statistical plots.
Plotly	Interactive, web-based plots.

Property	Values
figsize	(5,10)
color	• 'red' • '#FF0000' • (0.5, 0.2, 0.8)
linestyle	• linestyle = '-' linestyle = 'solid' • linestyle = '--' linestyle = 'dashed' • linestyle = ':' linestyle = 'dotted' • linestyle = '-.' linestyle = 'dash-dot' • linestyle = 'None'
Marker (Symbol that shows individual data points)	• 'o' (circle) • 's' (square) • '^' (triangle up) • 'v' (triangle down) • '+' (plus) • 'x' (cross) • '*' (star) • '.' (point)
linewidth	• 1 • 2 • 3 ...
alpha	• 0.0 (transparent) • 1.0 (opaque)
edgecolor	• 'red' • '#FF0000' • (0.5, 0.2, 0.8)
width	• 0.1 • 0.2 • 0.3 ...
align	• 'center' (default) • 'edge'
Bins (divide data into intervals)	5 → divides data into 5 intervals [5, 10, 15, 20] → custom intervals

density	• True • False
grid	• True • False
labels (without legend (), the labels won't appear)	List of labels ['Line1','Line2']
cmap	• 'viridis' • 'coolwarm' • 'plasma' • 'inferno'

xlabel	'String in X-axis'
ylabel	'String in Y-axis'
title	'String for Title of the Graph'
xticks / yticks	- List of positions [0,1,2,3] - Custom labels ['A','B','C']
xticks (positions) → sets tick positions xticks (positions, labels) → sets positions & custom labels	[1,2,3] ([1,2,3], ['A','B','C'])
xticks (positions) → sets tick positions xticks (positions, labels) → sets positions & custom labels	[1,2,3] ([1,2,3], ['A','B','C'])
legend	loc = 'best' loc = 'upper left' loc = 'upper right' loc = 'lower left' loc = 'lower right' loc = 'center'

Module 4

(1) NumPy (Numerical Python)

Library used for numerical calculations to solve complex scientific problems.

Installation → pip install numpy

a = np.array([1, 2, 3, 4, 5])

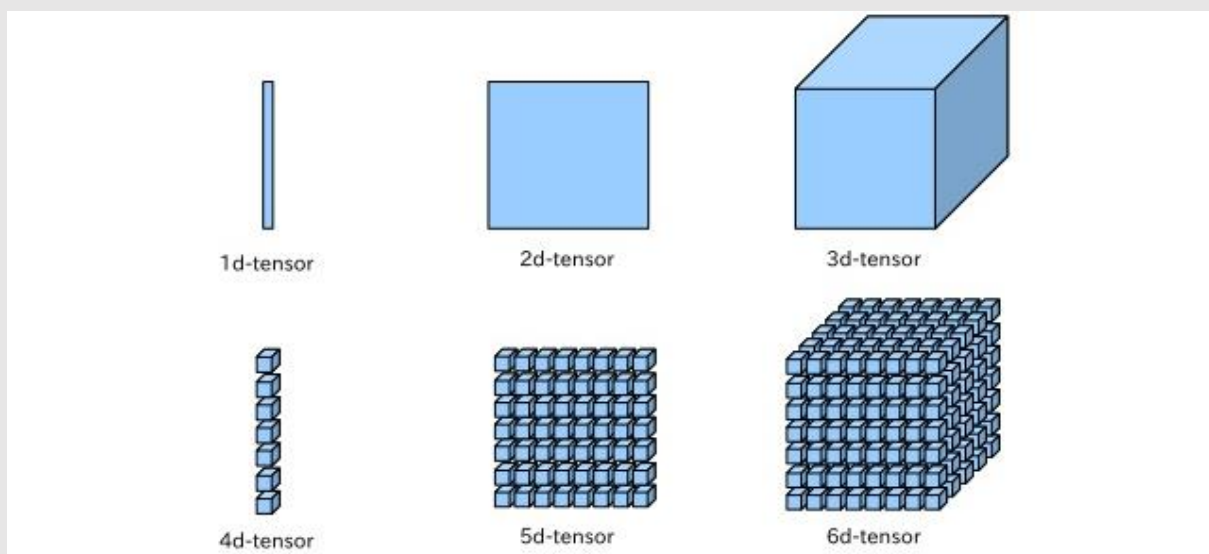
np. array (sequence)

- Function provided by the NumPy library to create NumPy array.
- Takes a list/sequence & returns a NumPy array (ndarray).

Data Structure → NumPy Array (ndarray) → a

Homogeneous collection of elements (all of the same data type) organized in multiple dimensions (1D, 2D, 3D, etc.), indexed by non-negative integers.

Dimensions	Name	Description	Example Use Case
0D	Scalar	A single element	np.array(5)
1D	Vector / Single Row	Stores elements in a single row.	[1, 2, 3, 4]
2D	Matrix	Stores elements in rows & columns.	[[1,2,3],[4,5,6]]
3D	Tensor	Stores elements in multiple layers.	Color image (height × width × RGB channels)
4D	Batch of Tensors	Array of 3D tensors (stacked along a new dimension)	Batch of images (batch_size, height, width, channels)



Why NumPy Arrays?

- Faster & more memory-efficient than lists (which are slower & use more memory).
- **Supports Element-wise Operations:** Operation done on each element individually, of arrays with the same shape.
- **Supports Vectorized Operations:** Operation done on whole arrays at once, without using loops.
- Provides powerful functions for linear algebra, statistics & random number generation.

Stacking

Combining multiple arrays along a new or existing axis to form a single array.

- **Vertical Stacking (vstack):** Stacks arrays **row-wise**, one on top of the other (adds along axis 0).
- **Horizontal Stacking (hstack):** Stacks arrays **column-wise**, side by side (adds along axis 1).

Standard normal distribution has

- Mean = 0 → most values are centered around 0.
- Standard deviation (spread) = 1 → most values lie roughly between -3 and +3.

****** Imagine a hill-shaped curve (a bell curve). Most numbers are near the top of the hill (0) & fewer numbers are far away from the top (like -2, -3, 2, 3).

Broadcasting Addition

```
# Original shapes:
# a6 → (2, 3) (2-D array with 2 rows, 3 columns)
# b6 → (3,) (1-D array with 3 elements)

# Compare dimensions from right → left:
# Last dimension: 3 (from a6) vs 3 (from b6) →  match
# Next dimension: 2 (from a6) vs no dimension (from b6)

# Missing dimension rule:
# If one array lacks a dimension, NumPy treats it as '1' in that position.
# So, b6 is treated as shape (1, 3).

# Broadcasting step:
# NumPy virtually expands b6 from (1,3) to (2,3) by repeating the row.
# b6 behaves like:
# [[1, 2, 3],
# [1, 2, 3]]

# Element-wise addition:
# [[1+1, 2+2, 3+3],
# [4+1, 5+2, 6+3]]
```

Function / Method	Description
np.array(sequence)	Creates a NumPy array from a list or other sequence.
np.zeros((m, n))	Creates an array of shape (m x n) filled with 0.
np.ones((m, n))	Creates an array of shape (m x n) filled with 1.
np.eye(n)	Creates an (n × n) identity matrix.
np.arange(start, stop, step)	Creates an array from start to stop-1 with a given step.
np.linspace(start, stop, num)	Creates an array of num evenly spaced numbers between start and stop (both included).
np.random.rand(rows, cols)	Creates an array of given shape with random floats in [0,1).
np.random.randint(start, stop, size=(m,n))	Creates an array of integers from start to stop-1 with shape (m x n).
np.random.randn(n)	Generates 'n' numbers from a standard normal distribution (mean=0, std=1).
array.shape a = np.array ([1, 2, 3]) b = np.array ([[1, 2], [4, 5]])	Returns the shape (rows, columns) of the array. a.shape → (3,) → 1-D array with 3 elements. b.shape → (2, 2) → 2-D array with 2-rows & 2-columns.
array.ndim	Returns the no. of dimensions of the array.
array.size	Returns the total no. of elements in the array.
array.dtype	Returns the data type of array elements.
array.itemsize	Returns the size (in bytes) of each element.
array[index]	Indexing: Access elements by position.
array[start:end]	Slicing: Access a subset of elements.
array.reshape(m, n)	Reshapes the array to the specified shape (m, n) without modifying its data.
arr.flatten()	Converts a multi-dimensional array into a 1D array.
array.T	Returns the transpose of the array.
array1 + array2	Adds two arrays element-wise.
array1 – array2	Subtracts two arrays element-wise.
array1 * array2	Multiplies two arrays element-wise.
array1 / array2	Divides two arrays element-wise.
array ** n	Raises each element to the power n.
Broadcasting	- NumPy automatically expands smaller arrays to match larger arrays' shapes so operations can be done element-wise without manually reshaping. 2D + 1D → 2D + (1D expanded to 2D) → result is 2D.
array.sum()	Returns the sum of all elements.
array.mean()	Returns the mean (average) of elements.
np.median(array)	Returns the median of array elements.
array.max()	Returns the maximum element.
array.min()	Returns the minimum element.
array.std() / np.std(array)	Returns the S.D. of elements.
array.var() / np.var(array)	Returns the variance of elements.
np.sqrt(array)	Returns the square root of each element.
np.exp(array)	Returns 'e' raised to the power of each element.
np.sin(array)	Returns the sine of each element (in radians).
np.log(array)	Returns the natural logarithm of each element.

<code>np.vstack((array1, array2))</code>	Vertically stacks arrays.
<code>np.hstack((array1, array2))</code>	Horizontally stacks arrays.
<code>np.vsplit(array, n)</code>	Splits an array vertically into n sub-arrays.
<code>np.hsplit(array, n)</code>	Splits an array horizontally into n sub-arrays.
<code>np.concatenate((arr1, arr2), axis)</code>	Combines two or more arrays along the specified axis (0/1) • 0 – Concatenate Vertically (like <code>v_stack</code>) • 1 – Concatenate Horizontally (like <code>h_stack</code>)
<code>np.dot (A, B) or A@B</code>	For 1D: dot product (sum of element-wise products). For 2D: Matrix multiplication.
<code>np.linalg.det(A)</code>	Returns determinant of matrix A.
<code>np.linalg.inv(A)</code>	Returns inverse of matrix A.
<code>np.save(filename, arr)</code> <code>np.save('arr.npy', A)</code>	Saves array to binary (.npy) file.
<code>np.load(filename)</code> <code>np.load('arr.npy')</code>	Loads array from binary (.npy) file.

```

import numpy as np

# 1. Creating Arrays
a = np.array([1, 2, 3, 4, 5])
# [1 2 3 4 5]

b = np.array([[1, 2, 3], [4, 5, 6]])
# [[1 2 3]
#  [4 5 6]]

zeros = np.zeros((2, 3))
# [[0. 0. 0.]
#  [0. 0. 0.]]

ones = np.ones((3, 2))
# [[1. 1.]
#  [1. 1.]
#  [1. 1.]]

I = np.eye(3)
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

arr1 = np.arange(1, 11, 2)
# [1 3 5 7 9]

arr2 = np.linspace(0, 1, 5)
# [0.    0.25 0.5   0.75 1.   ]

rand = np.random.rand(2, 3)
# [[0.99153946 0.07092729 0.06197796]
#  [0.00297777 0.07186913 0.9595701  ]]

rand_int = np.random.randint(1, 10, (3, 3))
# [[2 2 2]
#  [2 4 7]
#  [6 4 2]]

rand_norm = np.random.randn(5)
# [-0.10321885  0.4105985  0.14404357  1.45427351  0.76103773]

# 2. Attributes / Properties
print(b.shape)    # (2, 3)
print(b.ndim)     # 2
print(b.dtype)    # int64
print(b.size)     # 6
print(b.itemsize) # 8

```

```

# b.dtype is int64 → 64-bit integer.
# 1 byte = 8 bits, so 64 bits = 8 bytes.
# Therefore, b.itemsize = 8 (bytes per element).

# 3. Indexing & Slicing
arr4 = np.array([10, 20, 30, 40, 50])
print(arr4[0])      # 10
print(arr4[-2:])    # [40 50]

matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Second row:", matrix[1])      # [4 5 6]
print("Element (2nd row, 3rd col):", matrix[1,2]) # 6
print("First two rows:\n", matrix[:2]) # [[1 2 3],[4 5 6]]

# 4. Reshape & Manipulation
arr3 = np.arange(1, 10).reshape(3, 3)
print(arr3)
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]

print(arr3.T)
# [[1 4 7]
#  [2 5 8]
#  [3 6 9]]

arr = np.arange(1, 13)
arr2d = arr.reshape(3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

flat = arr2d.flatten()
# [ 1  2  3  4  5  6  7  8  9 10 11 12]

# 5. Arithmetic Operations
a1 = np.array([1, 2, 3, 4])
a2 = np.array([5, 6, 7, 8])
print(a1 + a2)      # [ 6  8 10 12]
print(a1 - a2)      # [-4 -4 -4 -4]
print(a1 * a2)      # [ 5 12 21 32]
print(a1 / a2)      # [0.2 0.33333333 0.42857143 0.5 ]
print(a1 ** 2)      # [ 1  4  9 16]

```

```

# 6. Broadcasting
a6 = np.array([[1,2,3],[4,5,6]])
b6 = np.array([1,2,3])
print("Broadcasted Addition:\n", a6+b6)
# [[2 4 6]
#   [5 7 9]]

# 7. Aggregate Functions
a3 = np.array([1, 2, 3, 4, 5])
print(a1.sum())      # 10
print(a1.mean())     # 2.5
print(np.median(a3)) # 3.0
print(a1.max())      # 4
print(a1.min())      # 1
print(a1.std())       # 1.118033988749895
print(np.std(a3))     # 1.4142135623730951
print(a1.var())       # 2.0
print(np.var(a3))     # 2.0

# 8. Mathematical Functions
print(np.sqrt(a3))    # [1.  1.41421356  1.73205081  2.  2.23606798]
print(np.exp(a3))     # [2.71828183  7.3890561  20.08553692  54.59815003
148.4131591]
print(np.sin(a3))     # [ 0.84147098  0.90929743  0.14112001 -0.7568025 -
0.95892427]
print(np.log(a3))     # [0.  0.69314718  1.09861229  1.38629436  1.60943791]

# 9. Stacking & Splitting
a5 = np.array([[1,2],[3,4]])
b5 = np.array([[5,6],[7,8]])

v_stack = np.vstack((a5,b5))
# [[1 2]
#   [3 4]
#   [5 6]
#   [7 8]]

h_stack = np.hstack((a5,b5))
# [[1 2 5 6]
#   [3 4 7 8]]

split_v = np.vsplit(v_stack,2)
# [array([[1,2],[3,4]]), array([[5,6],[7,8]])]

split_h = np.hsplit(h_stack,2)
# [array([[1,2],[3,4]]), array([[5,6],[7,8]])]

```

```

concat_axis0 = np.concatenate((a5, b5), axis=0) # Vertical
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

concat_axis1 = np.concatenate((a5, b5), axis=1) # Horizontal
# [[1 2 5 6]
#  [3 4 7 8]]

# 10. Linear Algebra
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.dot(a, b)
# 1*4 + 2*5 + 3*6 = 32

A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(np.dot(A,B))
# [[19 22]
#  [43 50]]

print(np.linalg.det(A))
# -2.0

print(np.linalg.inv(A))
# [[-2.   1. ]
#  [ 1.5 -0.5]]
# 11. File I/O
arr_save = np.array([1,2,3,4,5])
np.save('arr.npy', arr_save)

loaded = np.load('arr.npy')
print("Loaded array:", loaded)
# [1 2 3 4 5]

```

(2) Pandas

Library used for data analysis (examining and understanding data) & manipulation (organizing and modifying data).

Two Data Structures

- **Series** – **1D labeled array**, can hold any data type (mixed types allowed; numeric types may be upcast). When placed inside a Data-Frame, it behaves like a single column.
- **Data-Frame** – **2D table of data** with labeled rows & columns, capable of holding different types of data in each column & is composed of multiple Series as columns.

Installation → pip install pandas

inplace = True means the changes occur in the original DataFrame (no need to assign to a new variable).

Note: pandas automatically upcasts numeric columns to float if there is any missing value (NaN) in the column.

Series 1		Series 2		Series 3		Dataframe			
INDEX	DATA	INDEX	DATA	INDEX	DATA	INDEX	SERIES 1	SERIES 2	SERIES 3
0	A	0	1	0	[1, 2]	0	A	1	[1, 2]
1	B	1	2	1	A	1	B	2	A
2	C	2	3	2	1	2	C	3	1
3	D	3	4	3	(4, 5)	3	D	4	(4, 5)
4	E	4	5	4	{"a": 1}	4	E	5	{"a": 1}
5	F	5	6	5	6	5	F	6	6

Series

Category	Function / Attribute	Description
Creation	pd.Series(data, index, dtype, name, copy)	Create a Series.
	data	Values (list, array, dict, scalar, etc).
	index	Custom Row Labels.
	dtype	Force converts to any data type (int, float, string, etc.)
	name	Optional name for the Series.
	copy	Whether to make a copy of the data.
Attributes	s.dtype	Data type of the Series elements.
	s.shape	Shape of the Series.
	s.size	No. of elements in the Series.
	s.name	Name of the Series.
Basic Info	s.head(n)	Returns the first 'n' values (default 5).
	s.tail(n)	Returns the last 'n' values (default 5).
	s.index	Returns the index labels of the Series.
	s.index.tolist()	Converts the index labels to a Python list.
	s.values	Returns the Series data as a NumPy array.
	s.values.tolist()	Converts the Series data to a Python list.
Math / Statistics	s.sum()	Sum of all values (ignores NaN).
	s.mean()	Mean (average) of values.
	s.median()	Median of values.
	s.mode()	Mode of values.
	s.max()	Maximum value.
	s.min()	Minimum value.
	s.std()	Standard deviation.
	s.var()	Variance.
Selection / Access	s[index]	Access element by integer position or label.
	s[1:3]	Slice values using Python slicing.
	s.loc[label]	Access element by label.
	s.iloc[position]	Access element by integer position.
Cleaning / Handling Missing Values	s.dropna()	Remove missing (NaN) values.
	s.fillna(value)	Fill missing (NaN) values with a specific value.
	s.replace(old, new)	Replace specified value with another.
Other Useful Methods	s.value_counts()	Count frequency of each unique value.
	s.unique()	Return array of unique values.
	s.sort_values()	Sort Series by values.
	s.sort_index()	Sort Series by index.

```
import pandas as pd
import numpy as np

# Default Syntax of Series with default values
pd.Series(
    data=None,      # no data
    index=None,     # index will be 0, 1, 2, ...
    dtype=None,     # infer dtype from data
    name=None,      # no name
    copy=False      # does not copy data
)
```

```
# 1. Creation
```

```
s = pd.Series(
    data = [10, 20, 30, np.nan, 20, 10, 40],
    index = ["A", "B", "C", "D", "E", "F", "G"],
    dtype = "float",
    name = "Marks",
    copy = False
)
```

```
print("Series:\n", s)
```

```
# Series:
```

```
# A    10.0
```

```
# B    20.0
```

```
# C    30.0
```

```
# D     NaN
```

```
# E    20.0
```

```
# F    10.0
```

```
# G    40.0
```

```
# Name: Marks, dtype: float64
```

```
# 2. Attributes
```

```
print(s.dtype) # float64
```

```
print(s.shape) # (7,)
```

```
print(s.size)  # 7
```

```
print(s.name)  # Marks
```

```
# 3. Basic Info
```

```
print(s.head())
```

```
# A    10.0
```

```
# B    20.0
```

```
# C    30.0
```

```
# D     NaN
```

```
# E    20.0
```

```
# Name: Marks, dtype: float64
```

```

print(s.tail())
# C      30.0
# D      NaN
# E      20.0
# F      10.0
# G      40.0
# Name: Marks, dtype: float64

print(s.index) # Index(['A', 'B', 'C', 'D', 'E', 'F', 'G'], dtype='object')
print(s.index.tolist()) # ['A', 'B', 'C', 'D', 'E', 'F', 'G']
print(s.values) # [10. 20. 30. nan 20. 10. 40.]
print(s.values.tolist()) # [10.0, 20.0, 30.0, nan, 20.0, 10.0, 40.]

# 4. Math / Statistics
print(s.sum()) # 130.0
print(s.mean()) # 21.666666666666668
print(s.median()) # 20.0
print(s.mode()) # 0      10.0
                # 1      20.0
                # dtype: float64
print(s.max()) # 40.0
print(s.min()) # 10.0
print(s.std()) # 11.547005383792516
print(s.var()) # 133.33333333333334

# 5. Selection / Access
print("Access by index label 'C':", s['C'])
# Access by index label 'C': 30.0

print("Access by integer position 2:", s[2])
# Access by integer position 2:30.0

print("Slice values s[1:3]:\n", s[1:3])
# Slice values s[1:3]:
# B      20.0
# C      30.0
# Name: Marks, dtype: float64

print("Access using loc['C']:", s.loc['C'])
# Access using loc['C']: 30.0

print("Access using iloc[2]:", s.iloc[2])
# Access using iloc[2]: 30.0

```

```
# 6. Cleaning / Handling Missing Values
print(s.dropna())
# A    10.0
# B    20.0
# C    30.0
# E    20.0
# F    10.0
# G    40.0
# Name: Marks, dtype: float64

print(s.fillna(0))
# A    10.0
# B    20.0
# C    30.0
# D     0.0
# E    20.0
# F    10.0
# G    40.0
# Name: Marks, dtype: float64

print(s.replace(10, 99))
# A    99.0
# B    20.0
# C    30.0
# D     NaN
# E    20.0
# F    99.0
# G    40.0
# Name: Marks, dtype: float64

# 7. Other Useful Methods
print(s.value_counts())
# 20.0    2
# 10.0    2
# 30.0    1
# 40.0    1
# Name: Marks, dtype: int64

print(s.unique()) # [10. 20. 30. nan 40.]

print(s.sort_values())
# A    10.0
# F    10.0
# B    20.0
# E    20.0
# C    30.0
# G    40.0
# D     NaN # Name: Marks, dtype: float64
```

```
print(s.sort_index())  
# A      10.0  
# B      20.0  
# C      30.0  
# D       NaN  
# E      20.0  
# F      10.0  
# G      40.0  
# Name: Marks, dtype: float64
```

Data Frame

Category	Function / Attribute	Description
Creation	pd.DataFrame (data index columns dtype copy)	Create a DataFrame. List, dict, array, Series or Data-Frame. Row labels; Default is [0, 1, 2, ...] Column labels; default is inferred from data. Force all columns to a specific data type Whether to make a copy of the data.
Attributes	df.dtypes	Data types of each column.
	df.shape	Returns (rows, columns).
	df.size	Total no. of elements.
	df.index	Row labels.
	df.columns	Column labels.
Basic Info	df.head(n)	First n rows (default 5).
	df.tail(n)	Last n rows (default 5).
	df.info()	Summary of DataFrame including index, columns, dtypes.
	df.describe()	Statistical summary of numeric columns.
Selection / Access	df['col']	Select a single column (Series)
	df[['col1','col2']]	Select multiple columns
	df.loc[row_label]	Select row(s) by label
	df.iloc[row_index]	Select row(s) by integer position
	df.loc[:, 'col']	Select column(s) by label
	df.iloc[:, 0]	Select column(s) by integer position
Filtering (Conditional)	df[df['col'] > value]	Rows where condition is True
Add Columns	df['new-col'] = list of values	Add new column in original df.
Drop Column	df.drop('col', axis=1) df.drop('col', axis=1, inplace=True)	Remove column & returns a new df (we must assign it). Remove column from original df.
Update Column	df['col'] = df['col']. replace (old, new) df.loc[condition, column] = value	Replace values in column. Update specific cells based on a condition.
Rename Column	df.rename(columns = { 'old-col1': 'new-col1',	Rename old column name with new column name.

	'old-col2': 'new-col2'}, inplace=True)	
Sorting / Ranking	df.sort_values(by='col')	Sort by column values in ASC.
	df.sort_values(by='col', ascending = False)	Sort by column values in DESC.
	df.sort_index()	Sort by row index.
	df['rank'] = df['col'].rank(ascending = False)	Rank values in a column.
Math (Statistics)	df['col'].sum()	Sum of column
	df['col'].max()	Maximum
	df['col'].min()	Minimum
	df['col'].mean(skipna = True)	Mean of column
	df['col'].median()	Median
	df['col'].mode().tolist()	Mode
	df['col'].std()	Standard deviation
	df['col'].var()	Variance
Grouping (Aggregation)	df.groupby('col1')['col2'] .mean()	Groups the Data-Frame by col1 & calculates the mean of col2 for each group.
Merging (Joining)	pd.merge(df1, df2, on = 'col', how = 'inner/left/right/outer')	Merge Data-Frames
Concatenation	pd.concat([df1, df2], ignore_index=True)	Stack Data-Frames vertically
Other Useful Methods	df['col'].value_counts()	Frequency count of column values.
	df['col'].unique()	Unique values in column.
	df.drop_duplicates()	Removes duplicate rows from the DataFrame.
	df['col'] = df['col'].astype(DataType)	Force converts to numeric/text type.
	# For numeric/text types	
	df['col'] = pd.to_datetime(df['col'])	Force converts to date type.
	# For datetime type	

	<pre>df['col'] = df['col'].str.lower() df['col'] = df['col'].str.upper()</pre>	Converts text column to lower or upper case.
Detect / Handle / Drop Missing Values	<pre>df.isnull() df.isna() df['col'].isnull() df['col'].isna()</pre>	Shows True/False for each column. Shows True/False for particular column.
	<pre>df.fillna(value, inplace=True) df['col'].fillna(value, inplace=True)</pre>	Fills missing values in all columns. Fill missing values in particular column only.
	<pre>df.dropna() df.dropna(subset=['col'], inplace = True)</pre>	Remove rows with missing values from each column. Remove rows with missing values from particular column.
	<pre>pd.get_dummies(df[categorical-cols], drop_first=True)</pre>	- Converts each categorical column into binary (0/1) columns. - Each column represents one category. - Value = 1 if the row belongs to that category, 0 otherwise. - Drops the first category of each column to avoid redundancy (dummy variable trap). Example: For Fruit with categories Apple and Banana, only Fruit_Banana is created. If Fruit_Banana=0, it means Apple.

```
import pandas as pd
import numpy as np

# 1. Creation
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['New York', 'London', 'Paris', 'Tokyo']
}

df = pd.DataFrame(data)

print(df)
#      Name  Age   City
# 0   Alice   24 New York
# 1    Bob   27   London
# 2  Charlie   22    Paris
# 3   David   32    Tokyo

# 2. Attributes
print(df.dtypes)
# Name    object
# Age     int64
# City    object
# dtype: object

print(df.shape)    # (4, 3)
print(df.size)     # 12
print(df.index)    # RangeIndex(start=0, stop=4, step=1)
print(df.columns)  # Index(['Name', 'Age', 'City'], dtype='object')

# 3. Basic Info
print(df.head(3))
#      Name  Age   City
# 0   Alice   24 New York
# 1    Bob   27   London
# 2  Charlie   22    Paris

print(df.tail(2))
#      Name  Age   City
# 2  Charlie   22    Paris
# 3   David   32    Tokyo
```

```

df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 4 entries, 0 to 3
# Data columns (total 3 columns):
# #   Column  Non-Null Count  Dtype
# ---  -
# 0   Name    4 non-null      object
# 1   Age     4 non-null      int64
# 2   City    4 non-null      object
# dtypes: int64(1), object(2)

print(df.describe())
#                Age
# count    4.000000
# mean     26.250000
# std       4.112988
# min      22.000000
# 25%      23.250000
# 50%      25.500000
# 75%      28.500000
# max      32.000000

# 4. Selection / Access
print("Select 'Name' column:\n", df['Name'])
# 0      Alice
# 1       Bob
# 2    Charlie
# 3     David
# Name: Name, dtype: object

print("Select multiple columns:\n", df[['Name', 'Age']])
#      Name  Age
# 0  Alice   24
# 1   Bob   27
# 2 Charlie   22
# 3  David   32

print("Select first row by label (loc):\n", df.loc[0])
# Name      Alice
# Age         24
# City    New York
# Name: 0, dtype: object

print("Select second row by position (iloc):\n", df.iloc[1])
# Name      Bob
# Age         27
# City    London
# Name: 1, dtype: object

```

```

# 5. Filtering / Conditional
print(df['Age'] > 25)
# 0    False
# 1     True
# 2    False
# 3     True
# Name: Age, dtype: bool

print("People older than 25:\n", df[df['Age'] > 25]) # returns only the rows
where the condition (Age > 25) is True.
#   Name  Age  City
# 1   Bob   27 London
# 3  David   32  Tokyo

# 6. Add / Remove / Update Columns
df['Salary'] = [40000, 42000, 45000, 48000]
print(df)
#   Name  Age  City  Salary
# 0  Alice   24 New York  40000
# 1   Bob   27  London  42000
# 2 Charlie   22   Paris  45000
# 3  David   32   Tokyo  48000

# Case 1: Without inplace
new_df = df.drop('City', axis=1)
print(new_df)
#   Name  Age  Salary
# 0  Alice   24  40000
# 1   Bob   27  42000
# 2 Charlie   22  45000
# 3  David   32  48000

print(df)
#   Name  Age  City  Salary
# 0  Alice   24  Delhi  40000
# 1   Bob   27 London  42000
# 2 Charlie   22  Paris  45000
# 3  David   32  Tokyo  48000

# Case 2: With inplace
df.drop('City', axis=1, inplace=True)
print(df)
#   Name  Age  Salary
# 0  Alice   24  40000
# 1   Bob   27  42000
# 2 Charlie   22  45000
# 3  David   32  48000

```

```

# Update Salary
df['Salary'] = df['Salary'].replace(42000, 43000)

print( df)
#      Name  Age  Salary
# 0   Alice   24   40000
# 1    Bob    27   43000
# 2  Charlie   22   45000
# 3   David   32   48000

df.loc[df['City'] == 'Tokyo', 'Salary'] = 50000
print("Update Salary for Tokyo:\n", df)
#      Name  Age  City  Salary
# 0   Alice   24 New York  40000
# 1    Bob    27  London  42000
# 2  Charlie   22  Paris  45000
# 3   David   32  Tokyo  50000

# 7. Sorting / Ranking
df_sorted = df.sort_values(by='Salary', ascending=False)
print(df_sorted)
#      Name  Age  Salary
# 3   David   32   50000
# 2  Charlie   22   45000
# 1    Bob    27   42000
# 0   Alice   24   40000

df_sorted = df.sort_index(ascending=False)
print(df_sorted)
#      Name  Age  Salary
# 4  Charlie   22   45000
# 3   Alice   24   40000
# 2   David   32   50000
# 1    Bob    27   42000

df['Rank'] = df['Salary'].rank(ascending=False)
print(df)
#      Name  Age  Salary  Rank
# 0   Alice   24   40000   4.0
# 1    Bob    27   42000   3.0
# 2  Charlie   22   45000   2.0
# 3   David   32   50000   1.0

```

```

# 8. Math / Statistics
print(df['Salary'].sum())           # 177000
print(df['Salary'].max())           # 50000
print(df['Salary'].min())           # 40000
print(df['Salary'].mean())          # 44250.0
print(df['Salary'].median())        # 43500.0
print(df['Salary'].mode().tolist()) # [40000, 42000, 45000, 50000]
print(df['Salary'].std())            # 4330.127018922194
print(df['Salary'].var())            # 18750000.0

# 9(a). Grouping / Aggregation
data2 = {
    'Department': ['HR', 'IT', 'IT', 'HR', 'Finance'],
    'Employee': ['Anna', 'Bob', 'Charles', 'Diana', 'Eva'],
    'Salary': [40000, 50000, 45000, 42000, 52000]
}

df = pd.DataFrame(data2)

# Group by Department & calculate mean salary
grouped = df.groupby('Department')['Salary'].mean()
print("Average Salary by Department:\n", grouped)
# Department
# Finance      52000.0
# HR           41000.0
# IT           47500.0
# Name: Salary, dtype: float64

# 9(b). Merging
df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'ID': [1, 2, 4],
    'Salary': [50000, 60000, 70000]
})

merged_inner = pd.merge(df1, df2, on='ID', how='inner')
print(merged_inner)
#    ID    Name  Salary
# 0    1   Alice   50000
# 1    2    Bob    60000

```

```

left_merge = pd.merge(df1, df2, on='ID', how='left')
print(left_merge)
#      ID      Name  Salary
# 0     1    Alice  50000.0
# 1     2      Bob  60000.0
# 2     3  Charlie      NaN

right_merge = pd.merge(df1, df2, on='ID', how='right')
print(right_merge)
#      ID  Name  Salary
# 0     1  Alice   50000
# 1     2   Bob   60000
# 2     4   NaN   70000

outer_merge = pd.merge(df1, df2, on='ID', how='outer')
print(outer_merge)
#      ID      Name  Salary
# 0     1    Alice  50000.0
# 1     2      Bob  60000.0
# 2     3  Charlie      NaN
# 3     4      NaN  70000.0

# 9(c). Concatenation
df1 = pd.DataFrame({
    'Name': ['Alice', 'Bob'],
    'Age': [24, 27]
})

df2 = pd.DataFrame({
    'Name': ['Charlie', 'David'],
    'Age': [22, 32]
})

combined1 = pd.concat([df1, df2])
print(combined1)
#      Name  Age
# 0   Alice   24
# 1    Bob   27
# 0  Charlie   22
# 1   David   32

combined2 = pd.concat([df1, df2], ignore_index=True)
print(combined2)
#      Name  Age
# 0   Alice   24
# 1    Bob   27
# 2  Charlie   22
# 3   David   32

```

```

df1 = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'ID': [1, 2, 4],
    'Salary': [50000, 60000, 70000]
})

concatenate1 = pd.concat([df1, df2])
print(concatenate1)
#      ID      Name  Salary
# 0     1    Alice     NaN
# 1     2     Bob     NaN
# 2     3  Charlie     NaN
# 0     1     NaN  50000.0
# 1     2     NaN  60000.0
# 2     4     NaN  70000.0

concatenate2 = pd.concat([df1, df2], ignore_index=True)
print(concatenate2)
#      ID      Name  Salary
# 0     1    Alice     NaN
# 1     2     Bob     NaN
# 2     3  Charlie     NaN
# 3     1     NaN  50000.0
# 4     2     NaN  60000.0
# 5     4     NaN  70000.0

# 10. Other Useful Methods
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Bob'],
    'Age': [24, 27, 22, 32, 27],
    'Salary': [40000, 42000, np.nan, 48000, 42000]
}

df = pd.DataFrame(data)

print(df['Name'].value_counts())
# Bob      2
# Alice    1
# Charlie  1
# David    1
# Name: Name, dtype: int64

print(df['Age'].unique())
# [24 27 22 32]

```

```
# 11. Clean/Handle Missing Values
```

```
print(df.isnull())
```

```
#      Name    Age  Salary
# 0  False  False  False
# 1  False  False  False
# 2  False  False   True
# 3  False  False  False
# 4  False  False  False
```

```
print(df.dropna())
```

```
#      Name  Age  Salary
# 0  Alice   24 40000.0
# 1    Bob   27 42000.0
# 3  David   32 48000.0
# 4    Bob   27 42000.0
```

```
print(df['Salary'].fillna(0))
```

```
# 0    40000.0
# 1    42000.0
# 2         0.0
# 3    48000.0
# 4    42000.0
# Name: Salary, dtype: float64
```

(3) Data Acquisition, Cleaning & Preprocessing for Data Analysis

(1) Data Acquisition

Process of **collecting or importing data from different sources** so that it can be analyzed.

Common Sources & How to Access in Python

Source Type	Example	How to Access in Python
Local files	.csv, .xlsx, .json	pd.read_csv('file.csv'), pd.read_excel('file.xlsx')
Databases	MySQL SQLite PostgreSQL	Libraries like sqlite3 sqlalchemy pymysql
Web APIs	Weather Data Stock Data	requests pd.read_json(url)
Sensors / IoT Devices	Real-time temperature, traffic	Serial ports, MQTT, or streaming APIs
Web Scraping	Extracting data from websites.	BeautifulSoup or Scrapy

Why is Data Acquisition Important?

It is the foundation of any data analysis or machine learning project; correctly acquiring data ensures that the entire data science workflow is reliable and effective.

- **No data → no analysis:** You can't analyze or build models without data.
- The quality of data depends on how it's collected.
- The format & structure affect later steps (cleaning, modeling, etc.).
- Mistakes in acquisition (wrong encoding, missing columns) can lead to wrong insights.

```
# Example 1 – Creating a small dataset manually
data = {
    'Date': ['2025-01-01', '2025-01-02', '2025-01-03', '2025-01-04', '2025-01-05'],
    'Fruit': ['Apple', 'Banana', 'Apple', 'Banana', 'Apple'],
    'Quantity': [10, 15, np.nan, 20, 18],
    'Price_per_kg': [100, 80, 100, 90, 100]
}

df = pd.DataFrame(data)

# Example 2 – Reading data from a CSV file
df_csv = pd.read_csv('fruit_sales.csv')

# Example 3 – Reading data from a URL
url =
'https://raw.githubusercontent.com/datasets/fruit_sales/main/fruit_sales.csv'
df_url = pd.read_csv(url)
```

(2) Handling Missing Values

In real-world datasets, missing or blank values are very common and can appear in different forms depending on the type of data:

Data Type	Missing Representation	Example
Numeric columns	NaN (Not a Number)	[10, 15, NaN, 20]
Text columns	Blank "" or None	["Apple", "", "Banana"]
Dates / Categorical labels	Missing Entry (None)	["2025-01-01", None, "2025-01-03"]

Why Missing Values Matter?

1) Calculations Fail

If Quantity = [10, 15, NaN, 20], `df['Quantity'].mean()` → returns NaN unless handled.

2) Machine Learning Issues

- Most models **cannot handle missing values natively** (i.e., they don't work with NaN automatically).
- ML models require **complete numeric data**.
- Missing values must be **filled, imputed, or removed** before training.

3) Bias / Loss of Information

Dropping rows with missing values blindly can remove important patterns or reduce dataset size, affecting results.

Detect Missing Values (.isnull() or .isna())

```
# Count missing values in each column
print(df.isnull().sum())
# Date          0
# Fruit         0
# Quantity      1
# Price_per_kg  0
# dtype: int64
```

Handle Missing Values

```
# 1. Fill missing value with mean
df['Quantity'].fillna(df['Quantity'].mean(), inplace=True)

#
#   Date    Fruit  Quantity  Price_per_kg
# 0 2025-01-01  Apple    10.00         100
# 1 2025-01-02  Banana    15.00          80
# 2 2025-01-03  Apple   136.25         100
# 3 2025-01-04  Banana    20.00          90
# 4 2025-01-05  Apple   500.00         100

# 2. Fill missing value with median (better for outliers)
median_quantity = df['Quantity'].median(skipna=True)

df['Quantity'] = df['Quantity'].fillna(median_quantity)

#
#   Date    Fruit  Quantity  Price_per_kg
# 0 2025-01-01  Apple    10.0         100
# 1 2025-01-02  Banana    15.0          80
# 2 2025-01-03  Apple    17.5         100
# 3 2025-01-04  Banana    20.0          90
# 4 2025-01-05  Apple   500.0         100

# 3. Drop rows with missing values (if few & we don't want to fill)
df.dropna(subset=['Quantity'], inplace=True)

# Trade-off: Situation where gaining one advantage causes a disadvantage elsewhere.
• Dropping rows – Advantage: No missing data. Disadvantage: Reduces dataset size.
• Filling values – Advantage: Preserves dataset size. Disadvantage: May introduce bias.
```

(3) Feature Engineering

Process of creating new, meaningful features (columns) from existing data.

Goal

- To make the dataset more informative for analysis or machine learning.
- It helps models learn patterns faster & more accurately.

Why Feature Engineering is Important

Benefit	Explanation
Improves model performance	More relevant features help algorithms detect patterns
Enables better analysis	Makes it easier to see trends (e.g., higher sales on weekends)
Simplifies future calculations	Precomputes useful values like total price

Create New Features

```
# Create Total Price column
df['Total_Price'] = df['Quantity'] * df['Price_per_kg']
print(df[['Quantity', 'Price_per_kg', 'Total_Price']])
#      Quantity  Price_per_kg  Total_Price
# 0         10.0           100       1000.0
# 1         15.0            80       1200.0
# 2         17.5           100       1750.0
# 3         20.0            90       1800.0
# 4         17.5           100       1750.0

# Extract Day of the Week (.dt.day_name() → extracts weekday name from 'Date')
df['Day'] = df['Date'].dt.day_name()
print(df[['Date', 'Day']])
#      Date      Day
# 0 2025-01-01  Wednesday
# 1 2025-01-02   Thursday
# 2 2025-01-03    Friday
# 3 2025-01-04   Saturday
# 4 2025-01-05    Sunday

# Create Weekend Indicator
# df['Date'].dt.dayofweek >= 5 → Returns True for Saturday/Sunday
# (Mon-Sun → 0-6).
df['Is_Weekend'] = df['Date'].dt.dayofweek >= 5
print(df[['Date', 'Day', 'Is_Weekend']])
#      Date      Day  Is_Weekend
# 0 2025-01-01  Wednesday      False
# 1 2025-01-02   Thursday      False
# 2 2025-01-03    Friday      False
# 3 2025-01-04   Saturday       True
# 4 2025-01-05    Sunday       True
```

(4) Dimensionality Reduction

Reducing the no. of features (columns) in a dataset while keeping the most important information intact.

Goal: Simplify the dataset for faster analysis & better machine learning performance.

Benefits

- Speeds up computation.
- Reduces storage.
- Reduces overfitting in ML models.
- Makes data easier to visualize.

Example Dataset (After Feature Engineering)

Date	Fruit	Quantity	Price_per_kg	Total_Price	Day	Is_Weekend
2025-01-01	Apple	10.0	100	1000.0	Wednesday	False
2025-01-02	Banana	15.0	80	1200.0	Thursday	False
2025-01-03	Apple	17.5	100	1750.0	Friday	False
2025-01-04	Banana	20.0	90	1800.0	Saturday	True
2025-01-05	Apple	17.5	100	1750.0	Sunday	True

Problem

Not all columns are needed for every analysis.

For example, if we want to **predict Total_Price**, then:

- Day & Date might not affect the price directly,
- while Quantity & Price_per_kg are more important.

Why Dimensionality Reduction Matters

Reason	Explanation
Simpler Models	Fewer Features → Less risk of overfitting.
Faster Computation	Less data to process.
Noise Reduction	Removes irrelevant or redundant features.
Visualization	Easier to plot 2–3 features than 7.

Methods of Dimensionality Reduction

(a) Manual / Feature Selection

- This method involves **keeping only the most relevant columns (features)** for our task.
- **For example**, in machine learning or analysis, we might keep only the **numeric columns** that affect our target variable.

```
import pandas as pd

# Dataset after feature engineering
data = {
    'Date': ['2025-01-01', '2025-01-02', '2025-01-03', '2025-01-04', '2025-01-05'],
    'Fruit': ['Apple', 'Banana', 'Apple', 'Banana', 'Apple'],
    'Quantity': [10, 15, 17.5, 20, 17.5],
    'Price_per_kg': [100, 80, 100, 90, 100],
    'Total_Price': [1000, 1200, 1750, 1800, 1750],
    'Day': ['Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'],
    'Is_Weekend': [False, False, False, True, True]
}

df = pd.DataFrame(data)

# Dimensionality reduction: Keep only numeric columns
reduced_df = df[['Quantity', 'Price_per_kg', 'Total_Price']]

print("\nReduced dataset for analysis:")
print(reduced_df)
#   Quantity  Price_per_kg  Total_Price
# 0     10.0           100         1000
# 1     15.0            80         1200
# 2     17.5           100         1750
# 3     20.0            90         1800
# 4     17.5           100         1750
```

(b) PCA (Principal Component Analysis)

Statistical Technique

Method used to collect, organize, analyze, interpret & present data. **E.g.**, Mean, Median, Mode, S.D. / Variance, Correlation / Regression, Hypothesis Testing, PCA.

PCA (a method)

- Converts original features into new **uncorrelated features** called **principal components**.
- Retains **most of the variance** with fewer dimensions (When we reduce the no. of columns (features) in our dataset using PCA, the new features (principal components) still capture **almost all the important information** present in the original data. Variance indicates PCA which directions/features are important to keep.).
- Particularly **useful for large numeric datasets** to simplify analysis or speed up ML.

```
from sklearn.decomposition import PCA

# Select numeric columns
numeric_cols = df[['Quantity', 'Price_per_kg', 'Total_Price']]

# Initialize PCA to reduce to 2 dimensions
pca = PCA(n_components=2)

# Fit PCA and transform the data
pca_result = pca.fit_transform(numeric_cols)

# Display PCA result
print("\nPCA result (2 components):")
print(pca_result)

# n_components=2 → Reduce 3 numeric columns to 2 principal components
# Useful for visualization or speeding up ML

# Example output (numeric values will vary depending on dataset):
# [[-324.933  12.345]
#  [ -23.456  -7.890]
#  [  45.678   2.345]
#  [ 123.456  -1.234]
#  [ 179.255 -5.566]]
```

(c) Categorical Encoding

Process of converting categorical (text) data into numeric form so that machine learning algorithms can use it.

- **Why:** Most ML models can only work with numbers, not text.
- **How:** Each category is represented as a number or binary (0/1) column.
 - **Label Encoding:** Assigns a unique number to each category (Apple → 0, Banana → 1).
 - **One-Hot Encoding:** Creates separate binary columns for each category (Apple column, Banana column).

```
# Step 1: Select Categorical Columns
categorical_cols = ['Fruit', 'Day']

# Step 2: One-Hot Encoding
encoded_df = pd.get_dummies(df[categorical_cols], drop_first=True)
```

```
# Step 3: Output
Original dataset:
Fruit  Day
Apple  Wednesday
Banana Thursday
Apple  Friday
Banana Saturday
Apple  Sunday
```

One-Hot Encoded Dataset (drop_first=True):

Fruit_Banana	Day_Thursday	Day_Friday	Day_Saturday	Day_Sunday	Original Data
0	0	0	0	0	Apple, Wednesday
1	1	0	0	0	Banana, Thursday
0	0	1	0	0	Apple, Friday
1	0	0	1	0	Banana, Saturday
0	0	0	0	1	Apple, Sunday

(5) Data Quality & Cleaning Techniques

Purpose: Ensure dataset is accurate, complete, consistent & usable. Poor data leads to wrong analysis, bad ML predictions, or errors.

```
import pandas as pd

data = {
    'Date': ['2025-01-01', '2025-01-02', '2025-01-03', '2025-01-04', '2025-01-05', '2025-01-05'],
    'Fruit': ['Apple', 'Banana', 'Apple', 'Banana', 'Apple', 'Apple'],
    'Quantity': [10, 15, None, 20, 500, 500],
    'Price_per_kg': [100, 80, 100, 90, 100, 100],
    'Total_Price': [1000, 1200, 1000, 1800, 50000, 50000]
}

df = pd.DataFrame(data)

# Step 1: Detect and Remove Duplicates
df = df.drop_duplicates()

# Step 2: Handle Missing Values
df['Quantity'].fillna(df['Quantity'].mean(), inplace=True)
df['Fruit'].fillna('Unknown', inplace=True)

# Step 3: Correct Data Types
df['Quantity'] = df['Quantity'].astype(float)
df['Date'] = pd.to_datetime(df['Date'])

# Step 4: Handle Inconsistent Data
df['Fruit'] = df['Fruit'].str.lower()

# Step 5: Outlier Detection & Correction
Q1 = df['Quantity'].quantile(0.25)
Q3 = df['Quantity'].quantile(0.75)
IQR = Q3 - Q1
upper = Q3 + 1.5 * IQR
df.loc[df['Quantity'] > upper, 'Quantity'] = df['Quantity'].median()

# Step 6: Rename Columns
df.rename(columns={'Price_per_kg': 'PricePerKg', 'Total_Price': 'TotalPrice'},
inplace=True)
```

```
# Step 7: Ensure Consistency Across Columns
# df['TotalPrice'] = df['TotalPrice_check'] copies the values from the column
TotalPrice_check into the column TotalPrice.
# If TotalPrice already exists, its values are overwritten with the values
from TotalPrice_check.
df['TotalPrice_check'] = df['Quantity'] * df['PricePerKg']
df['TotalPrice'] = df['TotalPrice_check']
df.drop(columns=['TotalPrice_check'], inplace=True)

# Step 8: Final Clean Dataset
# Final clean dataset:
#      Date   Fruit  Quantity  PricePerKg  TotalPrice
# 0 2025-01-01  apple     10.0       100.0     1000.0
# 1 2025-01-02  banana     15.0        80.0     1200.0
# 2 2025-01-03  apple     16.7       100.0     1666.7
# 3 2025-01-04  banana     20.0        90.0     1800.0
# 4 2025-01-05  apple     16.7       100.0     1666.7
```

(4) EDA (Exploratory Data Analysis)

Process of **examining, cleaning, and visualizing data** to understand its main characteristics, detect patterns, spot anomalies, test hypotheses, and check assumptions using **summary statistics and graphical representations** before applying any machine learning or statistical modeling.

Steps in EDA

```
# Step 0: Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Data Loading and Overview
# Load the dataset into Python (usually with pandas) and check first few rows
df = pd.read_csv('data.csv')
print(df.head())

# Step 2: Understanding Data Structure
# Check shape, data types, and info of the dataset
print(df.shape)
print(df.info())
# Output: Column names, non-null count, and data types
print(df.columns.tolist())

# Step 3: Descriptive Statistics
# Summarize numeric data using mean, median, min, max, std, etc.
print(df.describe())
# Output: count, mean, std, min, 25%, 50%, 75%, max for numeric columns

# Step 4: Univariate Analysis
# Analyze single variables to see distribution or frequency

# Histogram of Salary
sns.histplot(df['Salary'], kde=True, color='skyblue')
plt.title("Salary Distribution")
plt.show()
# Output: Histogram showing salary distribution

# Bar chart for Department counts
sns.countplot(x='Department', data=df)
plt.title("Department Count")
plt.show()
# Output: Bar chart showing number of employees in each department
```

```
# Step 5: Bivariate & Multivariate Analysis
# Analyze relationships between two or more variables

# Scatter plot: Experience vs Salary colored by Promotion Status
sns.scatterplot(x='Experience', y='Salary', hue='Promotion_Status', data=df)
plt.title("Experience vs Salary vs Promotion")
plt.show()
# Output: Scatter plot showing relationship between Experience and Salary with
Promotion Status

# Correlation Heatmap for all numeric columns
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
# Output: Heatmap showing correlations between numeric columns

# Step 6: Outlier Detection
# Identify extreme values that deviate from normal range

# Boxplot to detect Salary outliers
sns.boxplot(x=df['Salary'])
plt.title("Boxplot - Outliers in Salary")
plt.show()
# Output: Boxplot showing outliers in Salary

# IQR method for identifying outliers
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['Salary'] < Q1 - 1.5*IQR) | (df['Salary'] > Q3 + 1.5*IQR)]
print(outliers)
# Output: DataFrame showing rows considered outliers

# Step 7: Correlation Analysis
# Measure linear relationship between selected numeric variables
corr = df[['Experience', 'Age', 'Salary', 'Performance_Score']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
# Output: Heatmap showing correlation values (1 = strong positive, -1 = strong
negative)
```

(1) EMPLOYEE EDA

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Step 1: CREATE DATASET
```

```
data = {
    'Emp_ID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'Finance', 'IT', 'HR', 'Finance', 'IT'],
    'Experience_Years': [2, 5, 8, 1, 3, 10, 6, 4, 12, np.nan],
    'Age': [25, 28, 35, 22, 26, 40, 31, 29, 45, 33],
    'Monthly_Salary': [30000, 50000, 80000, 25000, 32000, 90000, 60000, 40000, 95000, 0],
    'Performance_Score': [3.5, 4.2, 4.8, 2.9, 3.7, 4.9, 4.3, 3.6, 5.0, 4.1],
    'Promotion_Status': ['No', 'Yes', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No']
}
```

```
df = pd.DataFrame(data)
print(df)
```

#	Emp_ID	Department	Experience_Years	Age	Monthly_Salary	Performance_Score	Promotion_Status
# 0	101	HR	2.0	25	30000	3.5	No
# 1	102	IT	5.0	28	50000	4.2	Yes
# 2	103	Finance	8.0	35	80000	4.8	Yes
# 3	104	IT	1.0	22	25000	2.9	No
# 4	105	HR	3.0	26	32000	3.7	No
# 5	106	Finance	10.0	40	90000	4.9	Yes
# 6	107	IT	6.0	31	60000	4.3	Yes
# 7	108	HR	4.0	29	40000	3.6	No
# 8	109	Finance	12.0	45	95000	5.0	Yes
# 9	110	IT	NaN	33	0	4.1	No

```
# Step 2: DATA OVERVIEW
```

```
print(df.shape) # (10, 7)
```

```
print(df.info()) # Column Non-Null Count Dtype
```

```
Print(df.columns.tolist())
```

```
# ['Emp_ID', 'Department', 'Experience_Years', 'Age', 'Monthly_Salary', 'Performance_Score', 'Promotion_Status']
```

```
print(df.isnull().sum())
```

```
# Experience_Years    1
```

```
# Step 3: HANDLE MISSING VALUES
```

```
df['Experience_Years'] =  
df['Experience_Years'].fillna(df['Experience_Years'].mean())  
df['Monthly_Salary'] = df['Monthly_Salary'].replace(0,  
df['Monthly_Salary'].mean())
```

```
print(df)
```

#	Emp_ID	Department	Experience_Years	Age	Monthly_Salary	Performance_Score	Promotion_Status
# 0	101	HR	2.000000	25	30000.0	3.5	No
# 1	102	IT	5.000000	28	50000.0	4.2	Yes
# 2	103	Finance	8.000000	35	80000.0	4.8	Yes
# 3	104	IT	1.000000	22	25000.0	2.9	No
# 4	105	HR	3.000000	26	32000.0	3.7	No
# 5	106	Finance	10.000000	40	90000.0	4.9	Yes
# 6	107	IT	6.000000	31	60000.0	4.3	Yes
# 7	108	HR	4.000000	29	40000.0	3.6	No
# 8	109	Finance	12.000000	45	95000.0	5.0	Yes
# 9	110	IT	5.666667	33	50200.0	4.1	No

```
print(df.nunique())
```

# Emp_ID	10
# Department	3
# Experience_Years	10
# Age	10
# Monthly_Salary	10
# Performance_Score	10
# Promotion_Status	2

```
# Step 4: DESCRIPTIVE STATISTICS
```

```
print(df.describe())
```

#	Emp_ID	Experience_Years	Age	Monthly_Salary	Performance_Score
# count	10.00	10.000000	10.000000	10.000000	10.000000
# mean	105.50	5.666667	31.400000	55220.000000	4.100000
# std	3.03	3.496029	7.074210	25389.315338	0.683130
# min	101.00	1.000000	22.000000	25000.000000	2.900000
# 25%	103.25	3.250000	26.500000	34000.000000	3.625000
# 50%	105.50	5.333333	30.000000	50100.000000	4.150000
# 75%	107.75	7.500000	34.500000	75000.000000	4.675000
# max	110.00	12.000000	45.000000	95000.000000	5.000000

```

# Step 5: OUTLIER DETECTION

Q1 = df['Monthly_Salary'].quantile(0.25)
Q3 = df['Monthly_Salary'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df['Monthly_Salary'] < lower) | (df['Monthly_Salary'] > upper)]

# IMPORTANT:
# outliers store a filtered DataFrame that contains ONLY those rows that are
outliers in Monthly_Salary.

print("Outliers in Salary:", outliers[['Emp_ID', 'Monthly_Salary']])
# OUTPUT: Empty DataFrame → No Outliers on salary column

# Step 6: CORRELATION
numeric_cols =
df[['Experience_Years', 'Age', 'Monthly_Salary', 'Performance_Score']]
print(numeric_cols.corr())

#
#           Experience_Years      Age  Monthly_Salary  Performance_Score
# Experience_Years      1.000000  0.981002      0.956682      0.912578
# Age                  0.981002  1.000000      0.949879      0.879483
# Monthly_Salary       0.956682  0.949879      1.000000      0.886399
# Performance_Score    0.912578  0.879483      0.886399      1.000000

# Step 7: VIZUALIZATION
sns.heatmap(df.isnull(), cbar=False, cmap="Reds")
plt.title("Missing Values Heatmap")
plt.show()

corr = df[['Experience_Years', 'Age', 'Monthly_Salary',
'Performance_Score']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

sns.histplot(df['Monthly_Salary'], kde=True)
plt.title("Salary Distribution")
plt.show()

sns.countplot(x='Department', data=df)
plt.title("Department Count")
plt.show()

```

```
sns.boxplot(x=df['Performance_Score'])
plt.title("Performance Score Spread")
plt.show()

sns.boxplot(x=df['Monthly_Salary'])
plt.title("Boxplot - Outliers in Salary")
plt.show()

sns.scatterplot(x='Experience_Years', y='Monthly_Salary', data=df,
hue='Promotion_Status')
plt.title("Experience vs Salary vs Promotion")
plt.show()

sns.barplot(x='Department', y='Monthly_Salary', data=df)
plt.title("Department vs Salary")
plt.show()
```

(2) STUDENT EDA

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# STEP 1: CREATE DATASET
df = pd.DataFrame({
    'ID': [1, 2, 3, 4, 5, 6, 7, 8],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Ethan', 'Fiona', 'George', 'Hannah'],
    'Age': [16, 17, 16, 17, 16, 18, 17, 16],
    'Gender': ['F', 'M', 'M', 'F', 'M', 'F', 'M', 'F'],
    'Study_Hours': [4, 6, 2, 5, 3, 7, None, 6],
    'Attendance': [90, 80, 60, 95, 70, 85, 75, None],
    'Grade': [85, 78, 50, 88, 65, 92, 70, 90]
})

print(df)
```

	ID	Name	Age	Gender	Study_Hours	Attendance	Grade
# 0	1	Alice	16	F	4.0	90.0	85
# 1	2	Bob	17	M	6.0	80.0	78
# 2	3	Charlie	16	M	2.0	60.0	50
# 3	4	Diana	17	F	5.0	95.0	88
# 4	5	Ethan	16	M	3.0	70.0	65
# 5	6	Fiona	18	F	7.0	85.0	92
# 6	7	George	17	M	NaN	75.0	70
# 7	8	Hannah	16	F	6.0	NaN	90

```
# STEP 2: DATA OVERVIEW
print(df.info())
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 8 entries, 0 to 7
# Data columns (total 7 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   ID          8 non-null      int64
# 1   Name        8 non-null      object
# 2   Age         8 non-null      int64
# 3   Gender      8 non-null      object
# 4   Study_Hours 7 non-null      float64
# 5   Attendance  7 non-null      float64
# 6   Grade       8 non-null      int64
# dtypes: float64(2), int64(3), object(2)
# memory usage: 580.0 bytes
# None
```

```

print("Missing values:", df.isnull().sum())
# Study_Hours    1
# Attendance     1

print("Descriptive Statistics:", df.describe()) # for numerical columns
#
#      ID      Age  Study_Hours  Attendance      Grade
# count  8.000000   8.000000    7.000000    7.000000   8.000000
# mean   4.500000  16.625000    4.714286   79.285714  77.250000
# std    2.449490   0.744024    1.799471   12.051477  14.645819
# min    1.000000  16.000000    2.000000   60.000000  50.000000
# 25%    2.750000  16.000000    3.500000   72.500000  68.750000
# 50%    4.500000  16.500000    5.000000   80.000000  81.500000
# 75%    6.250000  17.000000    6.000000   87.500000  88.500000
# max    8.000000  18.000000    7.000000   95.000000  92.000000

# STEP 3: DATA REDUCTION
df = df.drop(['ID', 'Name'], axis=1)

print(df)
#      Age  Gender  Study_Hours  Attendance  Grade
# 0    16      F         4.0        90.0     85
# 1    17      M         6.0        80.0     78
# 2    16      M         2.0        60.0     50
# 3    17      F         5.0        95.0     88
# 4    16      M         3.0        70.0     65
# 5    18      F         7.0        85.0     92
# 6    17      M        NaN        75.0     70
# 7    16      F         6.0        NaN     90

# STEP 4: FEATURE ENGINEERING
df['Study_per_day'] = df['Study_Hours'] / df['Age']

print(df)
#      Age  Gender  Study_Hours  Attendance  Grade  Study_per_day
# 0    16      F         4.0        90.0     85      0.250000
# 1    17      M         6.0        80.0     78      0.352941
# 2    16      M         2.0        60.0     50      0.125000
# 3    17      F         5.0        95.0     88      0.294118
# 4    16      M         3.0        70.0     65      0.187500
# 5    18      F         7.0        85.0     92      0.388889
# 6    17      M        NaN        75.0     70          NaN
# 7    16      F         6.0        NaN     90      0.375000

```

```
# STEP 5: HANDLE MISSING VALUES
df['Study_Hours'] = df['Study_Hours'].fillna(df['Study_Hours'].mean())
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].median())
df['Study_per_day'] = df['Study_per_day'].fillna(df['Study_per_day'].median())
print(df)
```

#	Age	Gender	Study_Hours	Attendance	Grade	Study_per_day
# 0	16	F	4.000000	90.0	85	0.250000
# 1	17	M	6.000000	80.0	78	0.352941
# 2	16	M	2.000000	60.0	50	0.125000
# 3	17	F	5.000000	95.0	88	0.294118
# 4	16	M	3.000000	70.0	65	0.187500
# 5	18	F	7.000000	85.0	92	0.388889
# 6	17	M	4.714286	75.0	70	0.294118
# 7	16	F	6.000000	80.0	90	0.375000

```
print(df.describe()) # Describe After Cleaning
```

#	Age	Study_Hours	Attendance	Grade	Study_per_day
# count	8.000000	8.000000	8.000000	8.000000	8.000000
# mean	16.625000	4.714286	79.375000	77.250000	0.283446
# std	0.744024	1.665986	11.160357	14.645819	0.092572
# min	16.000000	2.000000	60.000000	50.000000	0.125000
# 25%	16.000000	3.750000	73.750000	68.750000	0.234375
# 50%	16.500000	4.857143	80.000000	81.500000	0.294118
# 75%	17.000000	6.000000	86.250000	88.500000	0.358456
# max	18.000000	7.000000	95.000000	92.000000	0.388889

```
# STEP 6: TRANSFORMATION
df['Grade_log'] = np.log(df['Grade'])
print(df)
```

#	Age	Gender	Study_Hours	Attendance	Grade	Study_per_day	Grade_log
# 0	16	F	4.000000	90.0	85	0.250000	4.442651
# 1	17	M	6.000000	80.0	78	0.352941	4.356709
# 2	16	M	2.000000	60.0	50	0.125000	3.912023
# 3	17	F	5.000000	95.0	88	0.294118	4.477337
# 4	16	M	3.000000	70.0	65	0.187500	4.174387
# 5	18	F	7.000000	85.0	92	0.388889	4.521789
# 6	17	M	4.714286	75.0	70	0.294118	4.248495
# 7	16	F	6.000000	80.0	90	0.375000	4.499810

```
# STEP 7: CORRELATION
print(df.corr())
```

#	Age	Study_Hours	Attendance	Grade	Study_per_day	Grade_log
# Age	1.000000	0.707107	0.397359	0.439145	0.632456	0.439145
# Study_Hours	0.707107	1.000000	0.605863	0.842894	0.992268	0.842894
# Attendance	0.397359	0.605863	1.000000	0.889292	0.612326	0.889292
# Grade	0.439145	0.842894	0.889292	1.000000	0.851678	1.000000
# Study_per_day	0.632456	0.992268	0.612326	0.851678	1.000000	0.851678
# Grade_log	0.439145	0.842894	0.889292	1.000000	0.851678	1.000000

```
# Step 8: VIZUALIZATION
# Gender Count
sns.countplot(x='Gender', data=df)
plt.title("Gender Count")
plt.show()

# Grade Distribution
sns.histplot(df['Grade'], kde=True)
plt.title("Grade Distribution")
plt.show()

# Study Hours vs Grade
sns.scatterplot(x='Study_Hours', y='Grade', data=df)
plt.title("Study Hours vs Grade")
plt.show()

# Attendance vs Grade
sns.scatterplot(x='Attendance', y='Grade', data=df)
plt.title("Attendance vs Grade")
plt.show()

# Correlation Heatmap
numeric_cols = df.select_dtypes(include=np.number)
sns.heatmap(numeric_cols.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

Module 5

[Matplotlib](#)

Library used for data visualization - it helps to create graphs & charts to show and understand our data easily.

- **Works with NumPy & Pandas:** Easy to use with numbers & tables.
- **Core Module:** matplotlib.pyplot – main tool to make graphs (like MATLAB).

Plot Type	Purpose / Use
Line plots	Show trends over time or continuous data.
Bar charts	Compare values across different categories.
Histograms	Show how data is distributed (frequency of values)
Scatter plots	Show the relationship between two things
Pie charts	Show parts of a whole
3D plots	Show data in three dimensions
Heatmaps	Show numbers in a table using colors

(1) Line Plot

Parameter	Description	Example
x	Values on x-axis	x = [1,2,3,4]
y	Values on y-axis	y = [10,20,15,25]
color / c	Line color	color = 'blue'
linestyle / ls	Line style ('-', '--', '-.', ':')	linestyle = '--'
linewidth / lw	Line thickness	linewidth = 2
marker	Symbol for points ('o','s','^','*')	marker = 'o'
markersize / ms	Size of marker	markersize = 8
markerfacecolor / mfc	Fill color of marker	mfc = 'yellow'
markeredgecolor / mec	Border color of marker	mec = 'red'
alpha	Transparency (0-1)	alpha = 0.5
label	Label for legend	label = 'Sales'

Marker	Shape
'o'	Circle
's'	Square
'^'	Triangle Up
'v'	Triangle Down
'D'	Diamond
'*'	Star
'+'	Plus
'x'	Cross
'.'	Point
'None'	No marker

Function	Purpose
plt.title()	Set chart title
plt.xlabel() / plt.ylabel()	Set axis labels
plt.legend()	Show legend
plt.grid()	Show/hide grid
plt.xlim() / plt.ylim()	Set axis limits
plt.xticks() / plt.yticks()	Change/set labels on x & y axis
plt.tight_layout()	Adjusts space to prevent overlap
plt.savefig(chart.jpg)	Save chart as image
plt.show()	Display chart

Color	Code / Name
Blue	'b' / 'blue'
Green	'g' / 'green'
Red	'r' / 'red'
Cyan	'c' / 'cyan'
Magenta	'm' / 'magenta'
Yellow	'y' / 'yellow'
Black	'k' / 'black'
White	'w' / 'white'

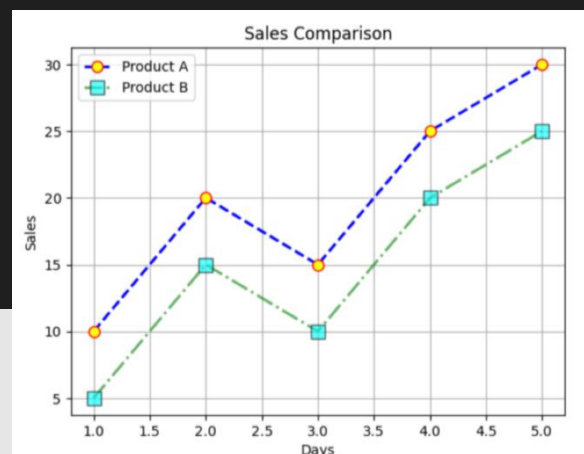
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [10, 20, 15, 25, 30]
y2 = [5, 15, 10, 20, 25]

plt.plot(x, y1, color='blue', linestyle='--', linewidth=2, marker='o',
         markersize=8, markerfacecolor='yellow', markeredgecolor='red',
         label='Product A')

plt.plot(x, y2, color='green', linestyle='-.', linewidth=2, marker='s',
         markersize=10, markerfacecolor='cyan', markeredgecolor='black',
         alpha=0.5, label='Product B')

plt.xlabel('Days')
plt.ylabel('Sales')
plt.title('Sales Comparison')
plt.grid(True)
plt.legend()
plt.show()
```



(2) Bar Chart

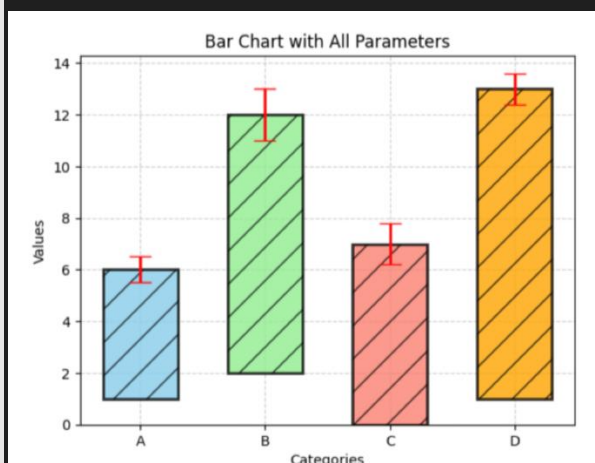
When log=True

- The **y-axis** becomes **logarithmic**, useful for **wide range values** (e.g., 10, 100, 1000).
- It helps to **visualize large & small values together**.

Parameter	Description	Example
x	Categories or x-values	['A', 'B', 'C']
height	Heights (y-values) of bars	[10, 20, 15]
width	Bar width (default 0.8)	0.5
bottom	Start point of bars	[0, 2, 1]
align	Bar alignment	'center' / 'edge'
color	Bar fill color	'skyblue'
edgecolor	Border color	'black'
linewidth	Border thickness	2
tick_label	Labels for bars	['A', 'B', 'C']
yerr	Error bar height ('l')	[0.5, 1.0, 0.8]
ecolor	Error bar color	'red'
capsize	End cap size of error bar ('-')	5
log	Use log scale	True / False
orientation	'vertical' or 'horizontal'	'vertical'
alpha	Transparency	0.8
hatch	Pattern on bars	'/'

```
# Example 1
import matplotlib.pyplot as plt
x = ['A', 'B', 'C', 'D']
y = [10, 15, 7, 12]

plt.bar(x, y, color='lightblue', edgecolor='black')
plt.title("Simple Bar Chart")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```



```

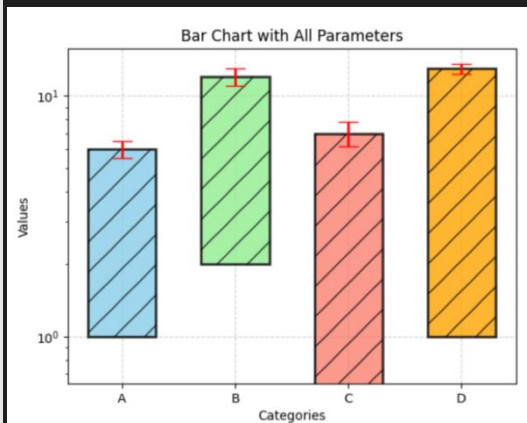
# Example 2 (log=True)
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(4)
y = [5, 10, 7, 12]
errors = [0.5, 1.0, 0.8, 0.6]

plt.bar(x, y,
        width=0.6,
        bottom=[1, 2, 0, 1],
        color=['skyblue', 'lightgreen', 'salmon', 'orange'],
        edgecolor='black',
        linewidth=2,
        tick_label=['A', 'B', 'C', 'D'],
        yerr=errors,
        ecolor='red',
        capsize=5,
        alpha=0.8,
        hatch='/')

plt.title("Bar Chart with All Parameters")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

```



(3) Histogram

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# -----
# GIVEN DATA
# -----
np.random.seed(24)
data = np.random.normal(350, 50, 150)
data = np.append(data, [800, 850, 900, 50, 40]) # Add outliers

print("Total Data Points:", len(data))

# -----
# (a) OUTLIER DETECTION BY 3 METHODS
# -----

# ===== 1. Standard Deviation Method =====
mean = np.mean(data)
std = np.std(data)

lower_sd = mean - 3*std
upper_sd = mean + 3*std

sd_outliers_idx = np.where((data < lower_sd) | (data > upper_sd))[0]
print("Standard Deviation Outliers Count:", len(sd_outliers_idx))
print("Indices:", sd_outliers_idx)

# ===== 2. IQR Method =====
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1

lower_iqr = Q1 - 1.5 * IQR
upper_iqr = Q3 + 1.5 * IQR

iqr_outliers_idx = np.where((data < lower_iqr) | (data > upper_iqr))[0]
print("IQR Outliers Count:", len(iqr_outliers_idx))
print("Indices:", iqr_outliers_idx)

# ===== 3. Z-SCORE Method =====
z_scores = stats.zscore(data)
z_outliers_idx = np.where(np.abs(z_scores) > 3)[0]

print("Z-score Outliers Count:", len(z_outliers_idx))
print("Indices:", z_outliers_idx)

```

```

# -----
# (b) OUTLIER HANDLING METHODS
# -----
# ===== (i) TRIMMING / REMOVING OUTLIERS =====
trimmed_data = data[np.abs(z_scores) <= 3] # Remove z-score outliers
print("Original size:", len(data))
print("After trimming:", len(trimmed_data))

# ===== (ii) Quantile-based Flooring and Capping =====
lower_cap = np.percentile(data, 5) # 5th percentile
upper_cap = np.percentile(data, 95) # 95th percentile

capped_data = np.copy(data)
capped_data[capped_data < lower_cap] = lower_cap
capped_data[capped_data > upper_cap] = upper_cap

print("Lower cap:", round(lower_cap,2))
print("Upper cap:", round(upper_cap,2))

# ===== (iii) Mean / Median Imputation =====
mean_val = np.mean(data)
median_val = np.median(data)

mean_imputed = np.copy(data)
median_imputed = np.copy(data)

mean_imputed[z_outliers_idx] = mean_val
median_imputed[z_outliers_idx] = median_val

# -----
# (c) REMOVE Z-SCORE OUTLIERS + COMPARISON STATS
# -----

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Std Dev:", np.std(data))

clean_data = trimmed_data # removed z-outliers

print("\n===== STATS AFTER CLEANING (Z-score removed) =====")
print("Mean:", np.mean(clean_data))
print("Median:", np.median(clean_data))
print("Std Dev:", np.std(clean_data))

```

```
# -----  
# PLOTTING (Boxplot + Histogram)  
# -----  
  
plt.figure(figsize=(12,5))  
  
plt.subplot(1,2,1)  
plt.boxplot(data)  
plt.title("Boxplot BEFORE Removing Outliers")  
  
plt.subplot(1,2,2)  
plt.boxplot(clean_data)  
plt.title("Boxplot AFTER Removing Outliers")  
  
plt.show()  
  
plt.figure(figsize=(12,5))  
  
plt.subplot(1,2,1)  
plt.hist(data, bins=20)  
plt.title("Histogram BEFORE Removing Outliers")  
  
plt.subplot(1,2,2)  
plt.hist(clean_data, bins=20)  
plt.title("Histogram AFTER Removing Outliers")  
  
plt.show()
```